Week 1

Web Basics

Experiment 1

Aim

To create a simple XHTML webpage explaining Web, WWW, Browser and Server.

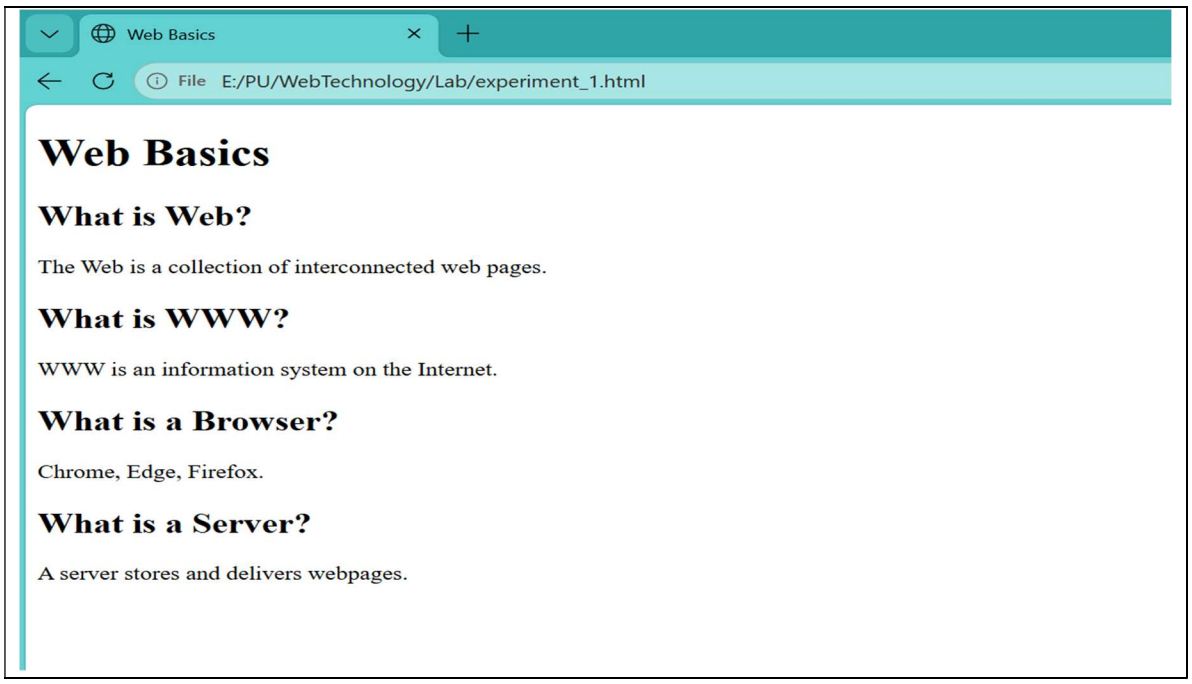
Procedure

- Create a new HTML file.
- Type the program.
- Save the file.
- Open in browser.
- Verify output.

Program (Source Code)

```
<!DOCTYPE html>
<head>
<title>Web Basics</title>
</head>
<body>
<h1>Web Basics</h1>
<h2>What is Web?</h2><p>The Web is a collection of interconnected web pages.</p>
<h2>What is WWW?</h2><p>WWW is an information system on the Internet.</p>
<h2>What is a Browser?</h2><p>Chrome, Edge, Firefox.</p>
<h2>What is a Server?</h2><p>A server stores and delivers webpages.</p>
</body>
</html>
```

Expected Output



Experiment 2

Aim

To design an XHTML webpage illustrating Internet architecture using text and images.

Procedure

- Create a new HTML file.
- Type the program.
- Save the file.
- Open in browser.
- Verify output.

Program (Source Code)

```
<!DOCTYPE html>
<head><title>Internet Architecture</title></head>
<body>
<h1>Internet Architecture</h1>
<p>The Internet connects clients and servers.</p>
<img src="web.png" width="600" height="350" alt="Internet Architecture"/>
<ul><li>Client</li><li>Internet</li><li>Server</li></ul>
</body></html>
```

Expected Output

Internet Architecture

The Internet connects clients and servers.



- Client
- Internet
- Server

XHTML Structure

Experiment 3

Aim

To create a basic XHTML document with proper syntax and structure.

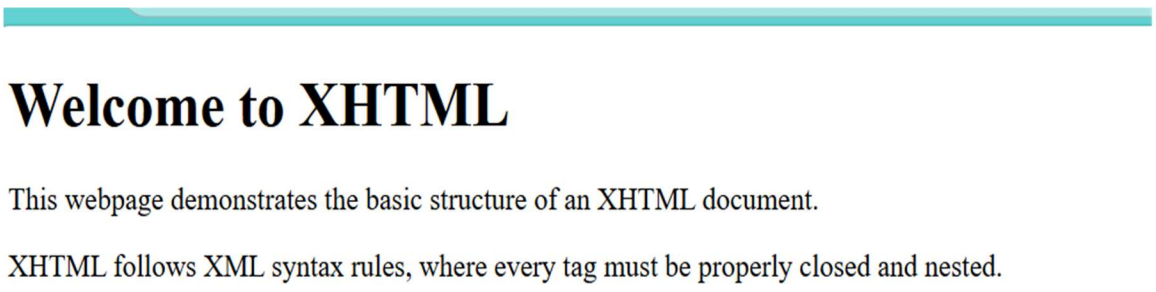
Procedure

- Create a new XHTML file.
- Type the XHTML program using the proper document structure.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<head>
  <title>Basic XHTML Structure</title>
</head>
<body>
  <h1>Welcome to XHTML</h1>
  <p>
    This webpage demonstrates the basic structure of an XHTML document.
  </p>
  <p>
    XHTML follows XML syntax rules, where every tag must be properly closed and nested.
  </p>
</body>
</html>
```

Expected Output



Experiment 4

Aim

To design an XHTML webpage using headings, paragraphs, and formatting tags.

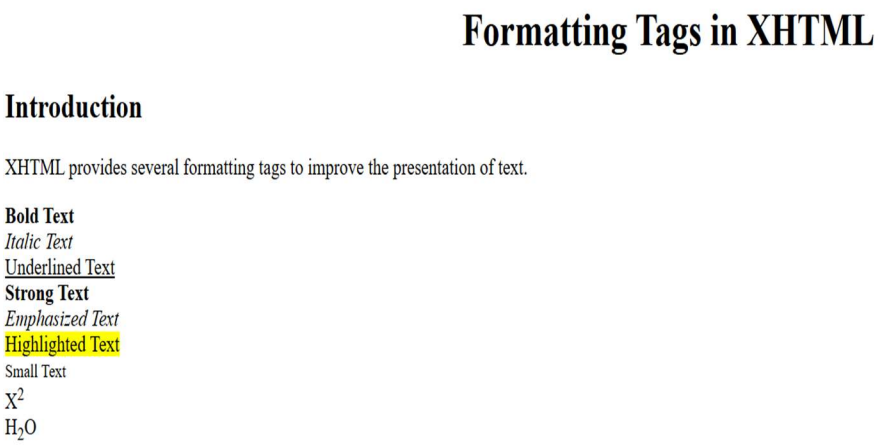
Procedure

- Create a new HTML file.
- Type the XHTML program using headings, paragraphs, and formatting tags.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html >
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>Formatting Tags</title>
</head>
<body>
<h1 align="center">Formatting Tags in XHTML</h1>
<h2>Introduction</h2><p>
XHTML provides several formatting tags to improve the presentation of text.
</p><p>
<b>Bold Text</b><br />
<i>Italic Text</i><br />
<u>Underlined Text</u><br />
<strong>Strong Text</strong><br />
<em>Emphasized Text</em><br />
<mark>Highlighted Text</mark><br />
<small>Small Text</small><br />
X<sup>2</sup><br />
H<sub>2</sub>O
</p></body></html>
```

Expected Output



Week 2
Lists and Links

Experiment 5

Aim

To create ordered, unordered, and definition lists using XHTML.

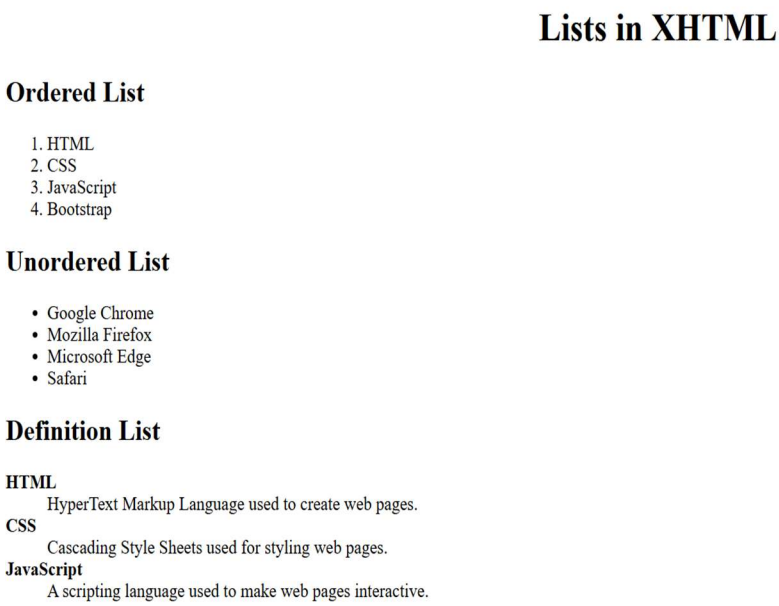
Procedure

- Create a new HTML file.
- Type the XHTML program using ordered, unordered, and definition list tags.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<head>
  <title>Lists in XHTML</title>
</head>
<body>
<h1 align="center">Lists in XHTML</h1>
<h2>Ordered List</h2>
<ol>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
  <li>Bootstrap</li>
</ol>
<h2>Unordered List</h2>
<ul>
  <li>Google Chrome</li>
  <li>Mozilla Firefox</li>
  <li>Microsoft Edge</li>
  <li>Safari</li>
</ul>
<h2>Definition List</h2>
<dl>
  <dt><b>HTML</b></dt>
  <dd>HyperText Markup Language used to create web pages.</dd>
  <dt><b>CSS</b></dt>
  <dd>Cascading Style Sheets used for styling web pages.</dd>
  <dt><b>JavaScript</b></dt>
  <dd>A scripting language used to make web pages interactive.</dd>
</dl>
</body>
</html>
```

Expected Output



Experiment 6

Aim

To create hyperlinks for navigation between multiple XHTML pages.

Procedure

- Create three XHTML files: **index.html**, **about.html**, and **contact.html**.
- Write the XHTML code for each webpage.
- Link the pages using the <a> (anchor) tag.
- Save all the files in the same folder.
- Open **index.html** in a web browser and verify the navigation between the pages.

Program (Source Code)

File 1: index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Home Page</title>
</head>
<body>
<h1>Welcome to My Website</h1>
<p>Select a page:</p>
<ul>
  <li><a href="about.html">About Us</a></li>
  <li><a href="contact.html">Contact Us</a></li>
</ul>
</body>
</html>
```

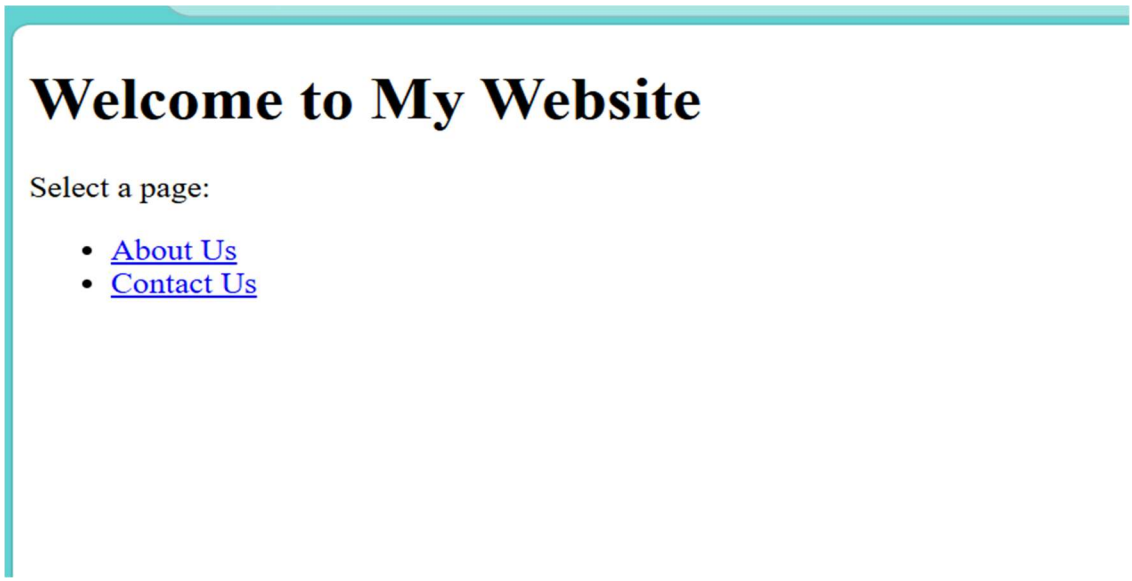
File 2: about.html

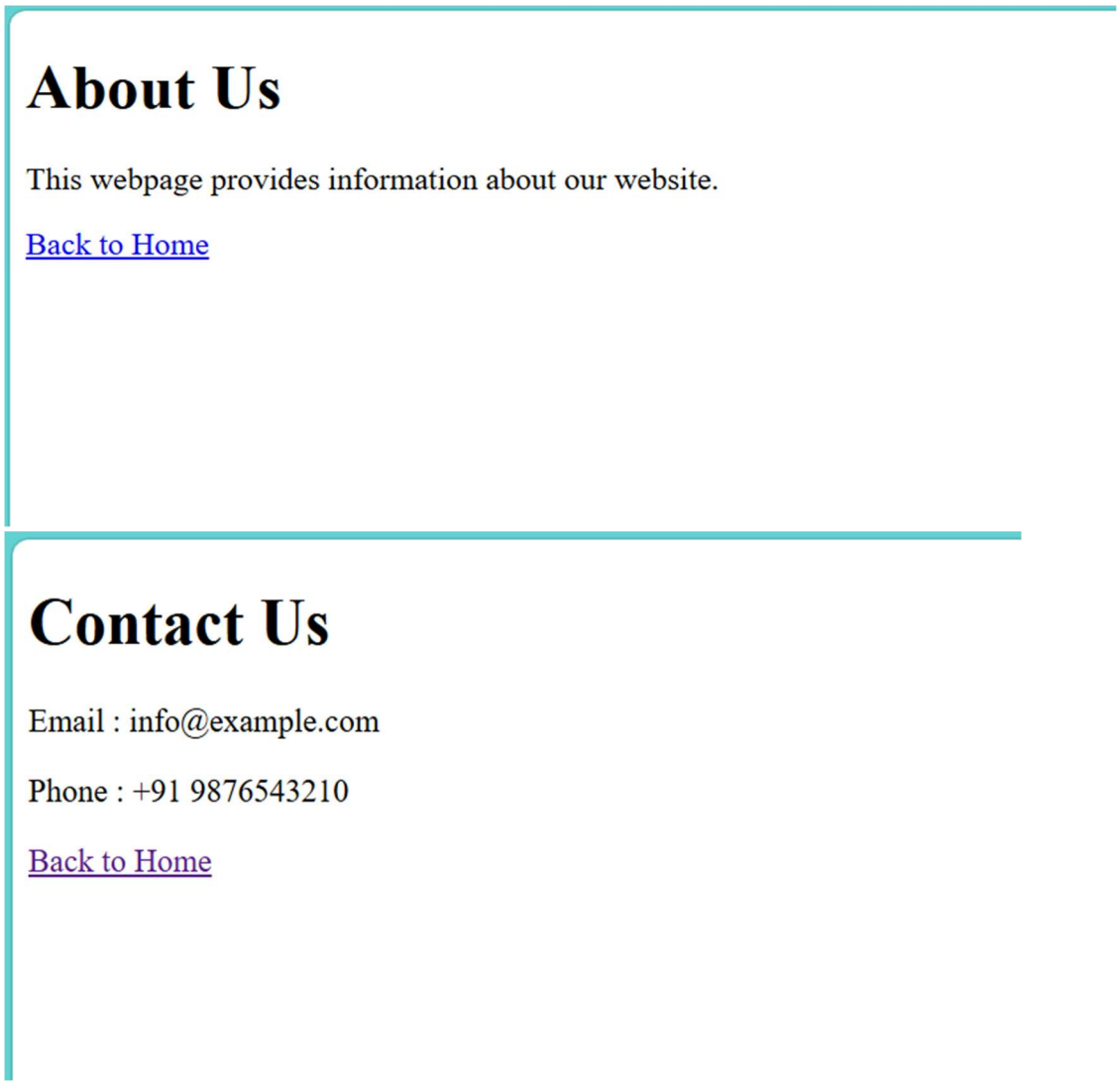
```
<!DOCTYPE html>
<html>
<head>
  <title>About</title>
</head>
<body>
<h1>About Us</h1>
<p>This webpage provides information about our website.</p>
<p><a href="index.html">Back to Home</a></p>
</body>
</html>
```

File 3: contact.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Contact</title>
</head>
<body>
<h1>Contact Us</h1>
<p>Email : info@example.com</p>
<p>Phone : +91 9876543210</p>
<p><a href="index.html">Back to Home</a></p>
</body>
</html>
```

Expected Output





Images and Tables

Experiment 7

Aim

To insert images into an XHTML webpage.

Procedure

- Save an image named **college.jpg** in the same folder as the XHTML file.
- Create a new XHTML file.
- Type the XHTML program using the <img> tag.
- Save the file with the **.html** extension.
- Open the file in a web browser and verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
```

```
<title>Images in XHTML</title>
</head>
<body>
<h1>My College</h1>
<p>Below is the image of our college campus.</p>
<img src="college.png" alt="College Image" width="400" height="250" />
</body>
</html>
```

Expected Output

My College

Below is the image of our college campus.



Experiment 8

Aim

To design a table with rows, columns, and formatting using XHTML.

Procedure

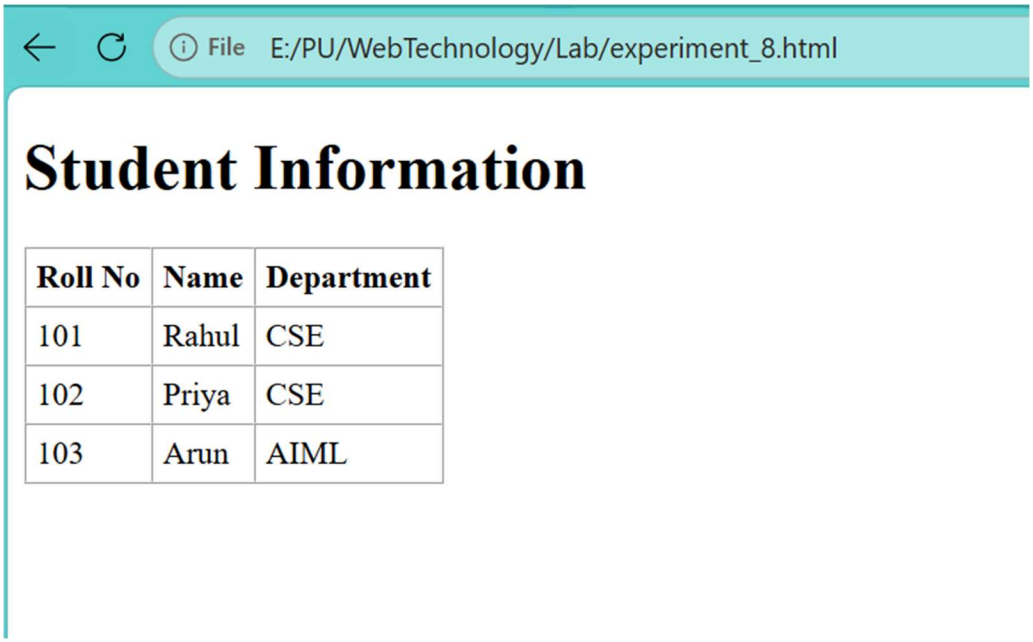
- Create a new XHTML file.
- Type the XHTML program using the <table>, <tr>, <th>, and <td> tags.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
<title>Student Table</title>
</head>
```

```
<body>
<h1>Student Information</h1>
<table border="1" cellpadding="5" cellspacing="0">
<tr>
<th>Roll No</th>
<th>Name</th>
<th>Department</th>
</tr>
<tr>
<td>101</td>
<td>Rahul</td>
<td>CSE</td>
</tr>
<tr>
<td>102</td>
<td>Priya</td>
<td>CSE</td>
</tr>
<tr>
<td>103</td>
<td>Arun</td>
<td>AIML</td>
</tr>
</table>
</body>
</html>
```

Expected Output



Week 3
Forms

Experiment 9

Aim

To create a simple XHTML form with text fields and buttons.

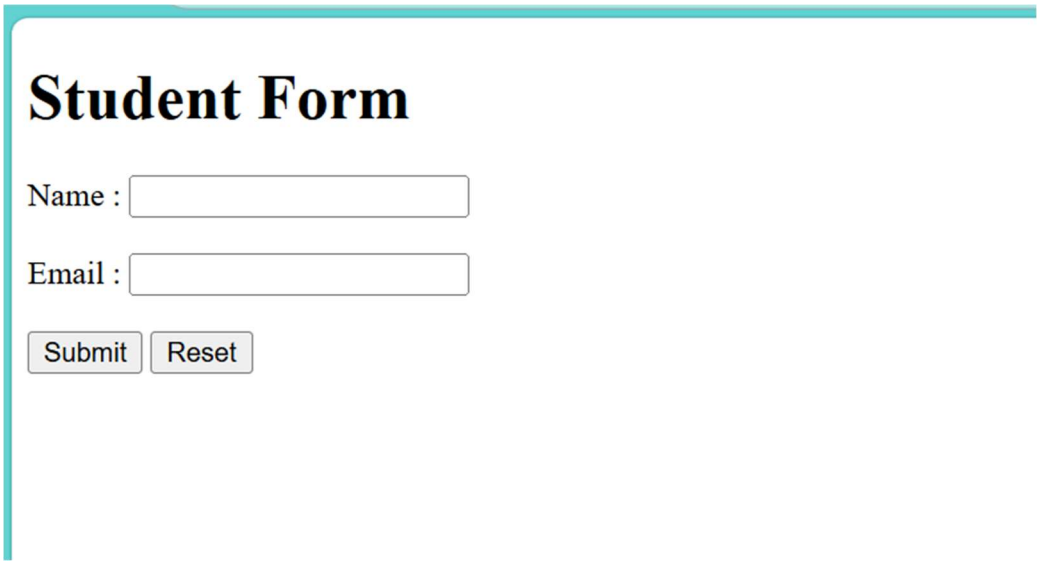
Procedure

- Create a new XHTML file.
- Type the XHTML program using the <form>, <input>, and <button> elements.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
   <title>Simple Form</title>
</head><body>
<h1>Student Form</h1>
<form>
Name :
<input type="text" name="name" /><br /><br />
Email :
<input type="text" name="email" /><br /><br />
<input type="submit" value="Submit" />
<input type="reset" value="Reset" />
</form>
</body></html>
```

Expected Output



Experiment 10

Aim

To design a registration form with validation requirements using XHTML.

Procedure

- Create a new XHTML file.
- Type the XHTML program using form elements.
- Add validation using the **required** attribute.
- Save the file with the **.html** extension.
- Open the file in a web browser and verify the validation.

Program (Source Code)

```
<!DOCTYPE html>
<html >
<head>
  <title>Registration Form</title>
</head>
<body>
<h1>Student Registration Form</h1>
<form>
Name :
<input type="text" name="name" required="required" />
<br /><br />
Email :
<input type="email" name="email" required="required" />
<br /><br />

Password :
<input type="password" name="password" required="required" />
<br /><br />
<input type="submit" value="Register" />
<input type="reset" value="Clear" />
</form>
</body>
</html>
```

Expected Output





Frames

Experiment 11

Aim

To create a webpage using frames in XHTML.

Procedure

- Create three HTML files: **left.html**, **right.html**, and **index.html**.
- Type the XHTML code for each file.
- Save all the files in the same folder.
- Open **index.html** in a web browser.
- Verify the output.

Program (Source Code)

File 1: index.html

```
<!DOCTYPE html >
<html >
<head>
  <title>Frames Example</title>
</head>
<frameset cols="30%,70%">
  <frame src="left.html" />
  <frame src="right.html" />
</frameset>

</html>
```

File 2: left.html

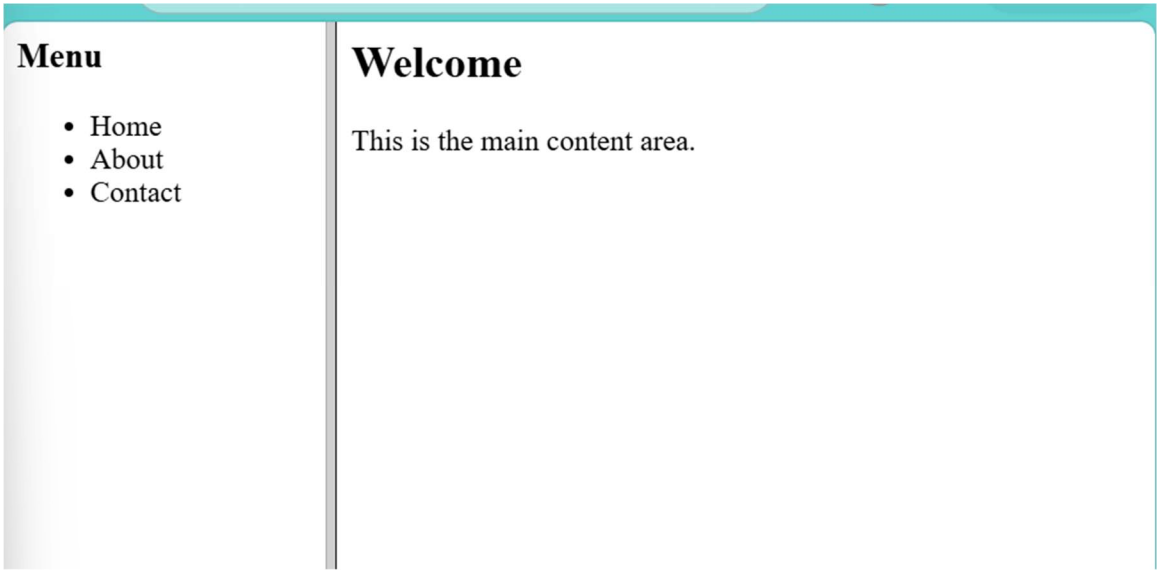
```
<html >
<body>
<h3>Menu</h3>
<ul>
  <li>Home</li>
  <li>About</li>
```

```
  <li>Contact</li>
</ul>
</body>
</html>
```

File 3: right.html

```
<html >
<body>
<h2>Welcome</h2>
<p>This is the main content area.</p>
</body>
</html>
```

Expected Output



Experiment 12

Aim

To design a multi-frame layout for a website using XHTML.

Procedure

- Create four HTML files: **header.html**, **menu.html**, **content.html**, and **index.html**.
- Type the XHTML code for each file.
- Save all the files in the same folder.
- Open **index.html** in a web browser.
- Verify the output.

Program (Source Code)

File 1: index.html

```
<!DOCTYPE html>
<html >
<head>
  <title>Multi Frame Layout</title>
```

```
</head>
<frameset rows="20%,80%">
  <frame src="header.html" />
  <frameset cols="30%,70%">
    <frame src="menu.html" />
    <frame src="content.html" />
  </frameset>
</frameset></html>
```

File 2: header.html

```
<html >
<body>
<h1 align="center">My Website</h1>
</body></html>
```

File 3: menu.html

```
<html >
<body>
<h3>Menu</h3>
<ul>
  <li>Home</li>
  <li>Gallery</li>
  <li>Contact</li>
</ul>
</body></html>
```

File 4: content.html

```
<html >
<body>
<h2>Welcome</h2>
<p>Welcome to our website.</p>
</body></html>
```

Expected Output



Week 4
CSS Basics

Experiment 13

Aim

To apply inline and internal CSS styles to an XHTML webpage.

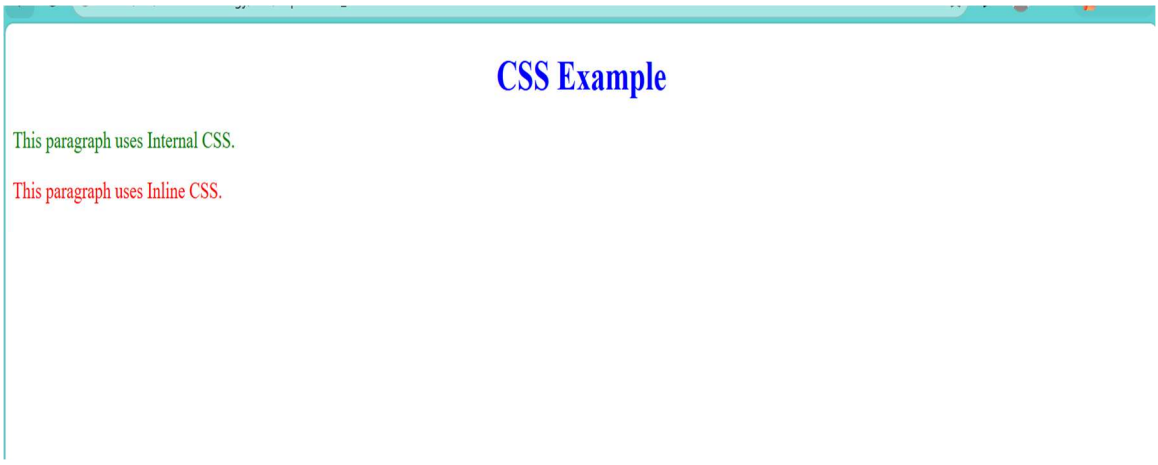
Procedure

- Create a new XHTML file.
- Type the XHTML program using inline and internal CSS.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html><head>
  <title>Inline and Internal CSS</title>
  <style>
    h1 {
      color: blue;
      text-align: center;
    }
    p {
      color: green;
      font-size: 18px;
    }
  </style>
</head><body>
<h1>CSS Example</h1>
<p>This paragraph uses Internal CSS.</p>
<p style="color:red;">This paragraph uses Inline CSS.</p>
</body></html>
```

Expected Output



Experiment 14

Aim

To create an external stylesheet and apply it to multiple XHTML pages.

Procedure

- Create three files: **style.css**, **page1.html**, and **page2.html**.
- Write the CSS code in **style.css**.
- Link the stylesheet to both XHTML pages.
- Save all the files in the same folder.
- Open the webpages in a web browser and verify the output.

Program (Source Code)

File 1: style.css

```
body{
   background-color: lightyellow;
   font-family: Arial;
}
h1{
   color: blue;
}
p{
   color: green;
}
```

File 2: page1.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Page 1</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>

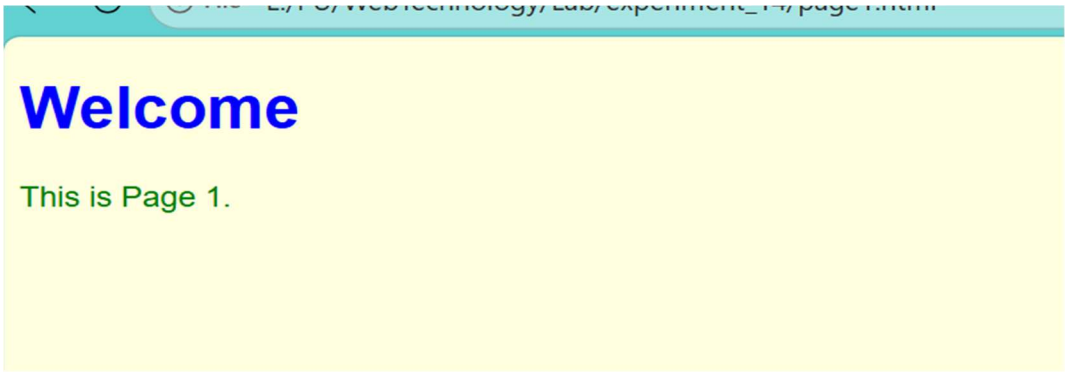
<h1>Welcome</h1>
<p>This is Page 1.</p>
</body>
</html>
```

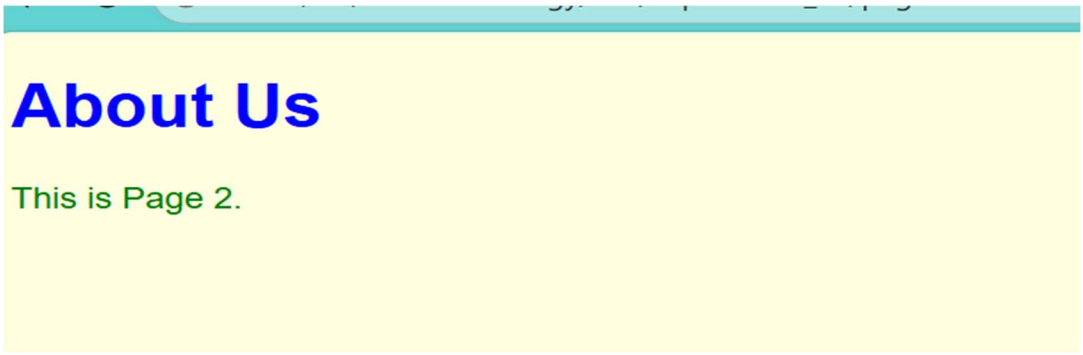
File 3: page2.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Page 2</title>
   <link rel="stylesheet" href="style.css">
</head>
```

```
<body>
<h1>About Us</h1>
<p>This is Page 2.</p>
</body>
</html>
```

Expected Output





Selectors

Experiment 15

Aim

To use element, class, and ID selectors in CSS for styling an XHTML webpage.

Procedure

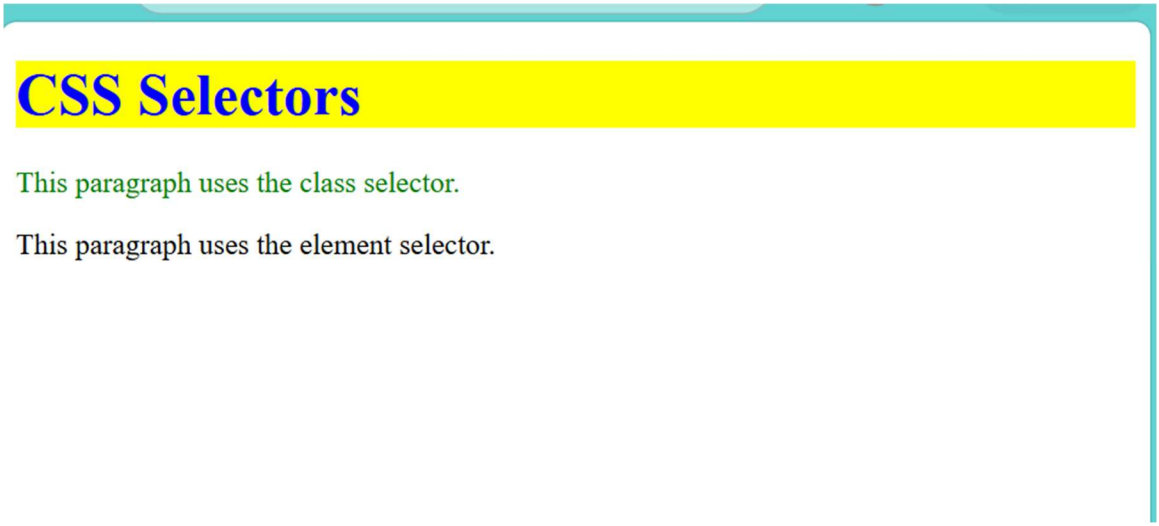
- Create a new HTML file.
- Type the XHTML program using element, class, and ID selectors.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>

<head>
  <title>CSS Selectors</title>
  <style>
    h1{
      color: blue;
    }
    .text{
      color: green;
    }
    #title{
      background-color: yellow;
    }
  </style>
</head>
<body>
<h1 id="title">CSS Selectors</h1>
<p class="text">This paragraph uses the class selector.</p>
<p>This paragraph uses the element selector.</p>
</body>
</html>
```

Expected Output



Experiment 16

Aim

To apply multiple CSS selectors for styling an XHTML webpage.

Procedure

- Create a new HTML file.
- Type the XHTML program using multiple CSS selectors.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Multiple Selectors</title>
  <style>

    h1, h2{
      color: blue;
    }
    p, li{
      color: green;
    }
  </style>
</head>
<body>
<h1>Welcome</h1>
<h2>Web Technology Lab</h2>
<p>This is a sample paragraph.</p>
```

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
</body>
</html>
```

Expected Output



Week 5
Font and Text

Experiment 17

Aim

To apply CSS font properties such as font size, font style, and font family to an XHTML webpage.

Procedure

- Create a new HTML file.
- Type the XHTML program using CSS font properties.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html><head>
```

```
  <title>Font Properties</title>
  <style>
    h1{
      font-family: Arial;
      font-size: 30px;
    }
    p{
      font-family: Times New Roman;
      font-size: 18px;
      font-style: italic;
    }
  </style></head>
<body>
<h1>Web Technology Lab</h1>
<p>This paragraph demonstrates CSS font properties.</p></body></html>
```

Expected Output



Experiment 18

Aim

To design a styled XHTML webpage using CSS text formatting properties.

Procedure

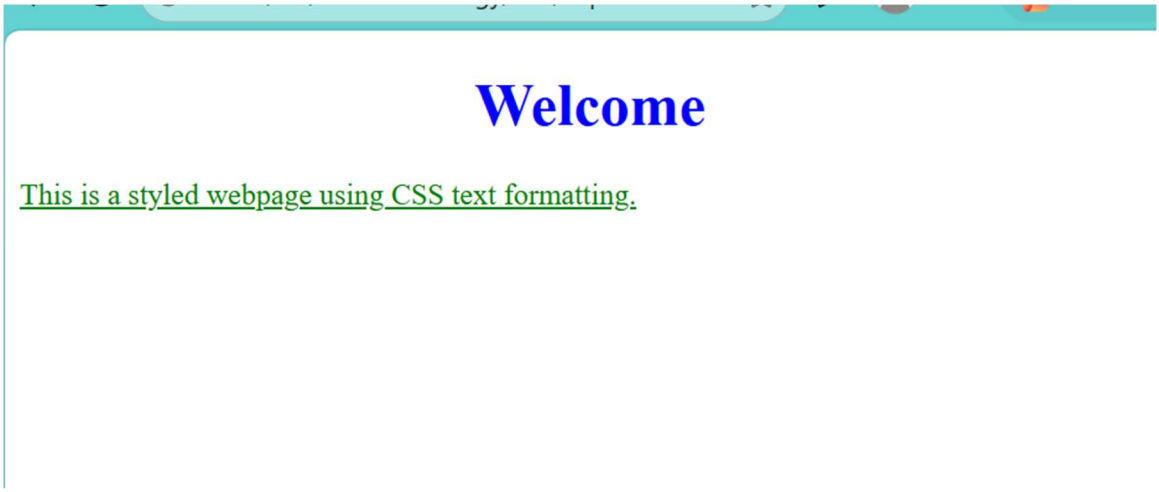
- Create a new HTML file.
- Type the XHTML program using CSS text formatting properties.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Text Formatting</title>
  <style>
```

```
    h1{
      color: blue;
      text-align: center;
    }
    p{
      color: green;
      text-decoration: underline;
    }
  </style>
</head>
<body>
<h1>Welcome</h1>
<p>This is a styled webpage using CSS text formatting.</p>
</body>
</html>
```

Expected Output



Box Model

Experiment 19

Aim

To demonstrate the use of CSS margin, padding, and border properties.

Procedure

- Create a new HTML file.
- Type the XHTML program using CSS margin, padding, and border properties.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

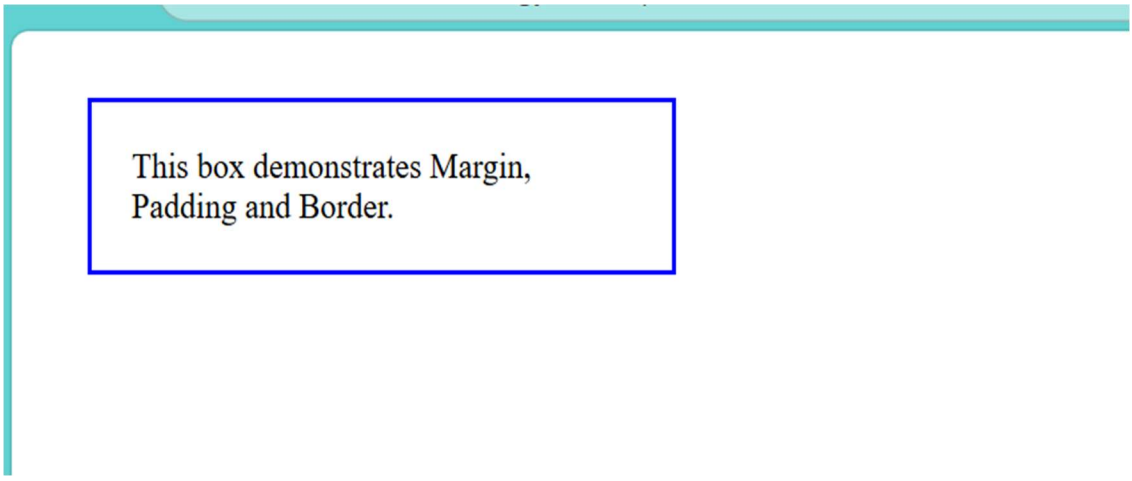
Program (Source Code)

```
<!DOCTYPE html>
```

```
<html>
<head>
  <title>Margin, Padding and Border</title>

  <style>
    div{
      border:2px solid blue;
      padding:20px;
      margin:30px;
      width:250px;
    }
  </style>
</head>
<body>
<div>
This box demonstrates Margin, Padding and Border.
</div>
</body></html>
```

Expected Output



Experiment 20

Aim

To design a simple webpage layout using CSS Box Model concepts.

Procedure

- Create a new HTML file.
- Type the XHTML program using CSS Box Model properties.
- Save the file with the .html extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Box Model Layout</title>
```

```
  <style>
    .box{
      width:300px;
      border:2px solid black;
      padding:15px;
      margin:20px;
      background-color:lightblue;
    }
  </style>
</head>
<body>
<div class="box">
<h2>Student Details</h2>
<p>Name : Rahul</p>
<p>Department : CSE</p>
</div>
</body>
</html>
```

Expected Output



Week 6
Positioning

Experiment 21

Aim

To demonstrate the use of static and relative positioning in CSS.

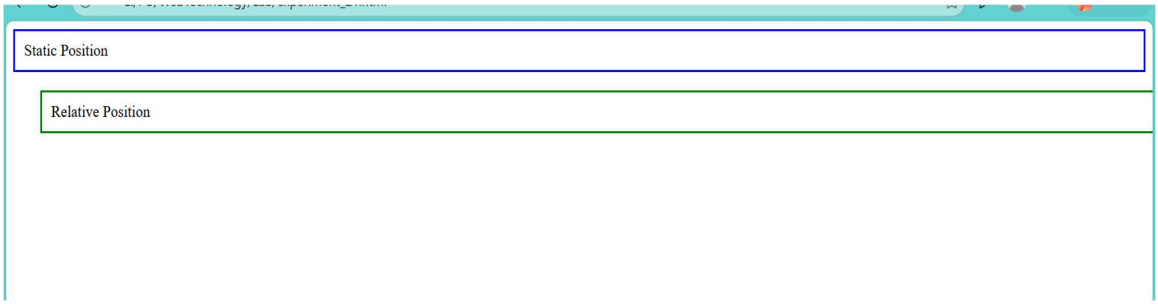
Procedure

- Create a new HTML file.
- Type the XHTML program using static and relative positioning.
- Save the file with the .html extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Static and Relative Positioning</title>
  <style>
    .static{
      border:2px solid blue;
      padding:10px;
    }
    .relative{
      border:2px solid green;
      padding:10px;
      position:relative;
      left:30px;
    }
  </style>
</head>
<body>
<div class="static">
Static Position
</div>
<br>
<div class="relative">
Relative Position
</div></body></html>
```

Expected Output



Experiment 22

Aim

To design a simple webpage layout using absolute and fixed positioning.

Procedure

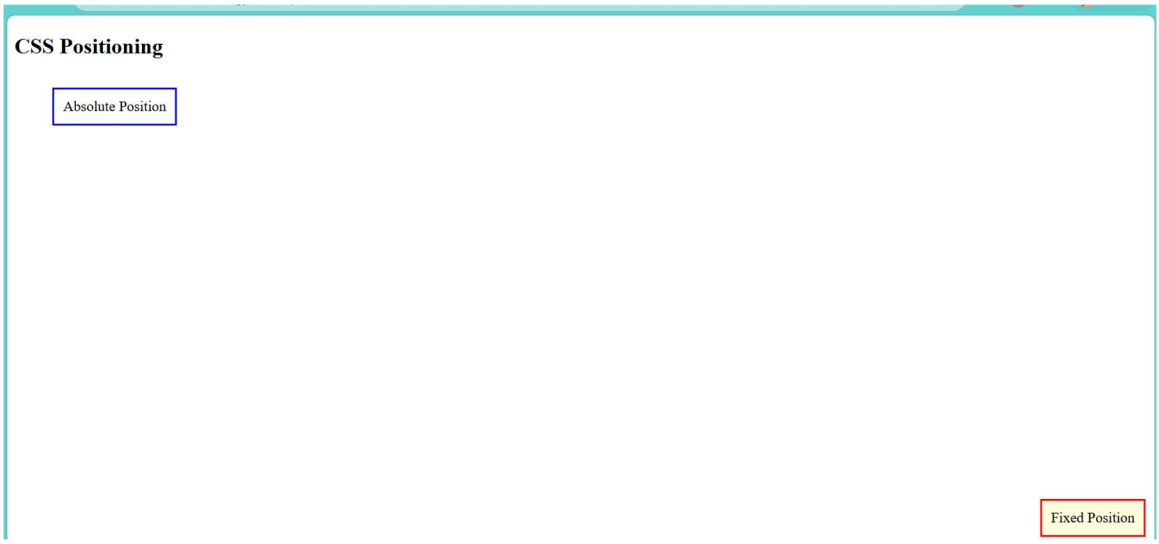
- Create a new HTML file.
- Type the XHTML program using absolute and fixed positioning.
- Save the file with the .html extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Absolute and Fixed Positioning</title>
  <style>
    .absolute{
      position:absolute;
      top:80px;
      left:50px;
      border:2px solid blue;
      padding:10px;
    }
    .fixed{
      position:fixed;
      bottom:10px;
      right:10px;
      border:2px solid red;
      padding:10px;
      background-color:lightyellow;
    }

  </style>
</head>
<body>
<h2>CSS Positioning</h2>
<div class="absolute">
Absolute Position
</div>
<div class="fixed">
Fixed Position
</div>
</body>
</html>
```

Expected Output



Floating & Layout

Experiment 23

Aim

To apply CSS float properties in an XHTML webpage.

Procedure

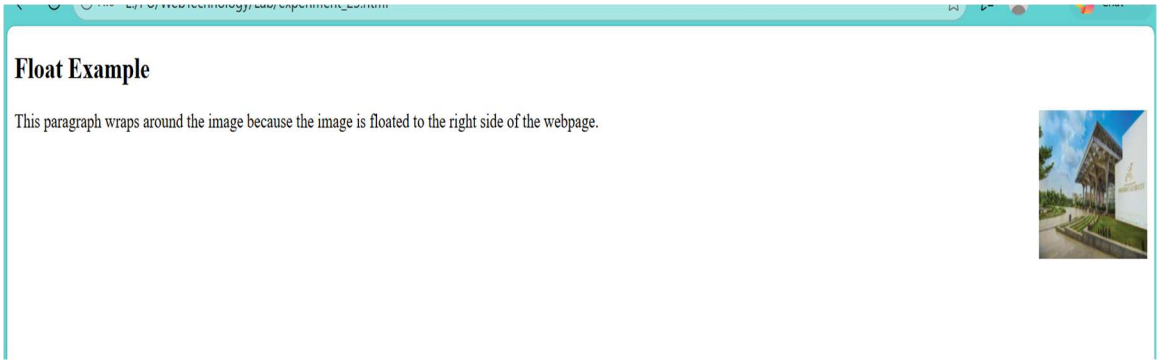
- Create a new HTML file.
- Type the XHTML program using the CSS float property.
- Save the file with the .html extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Float Property</title>
  <style>
    img{
      float:right;
      width:120px;
      height:120px;
    }
  </style>
</head>
<body>
<h2>Float Example</h2>
<img src="college.jpg" alt="College Image" />
<p>
This paragraph wraps around the image because the image is floated to the right side of the
webpage.
```

</p>
</body>
</html>

Expected Output



Experiment 24

Aim

To create a simple multi-column layout using the CSS float property.

Procedure

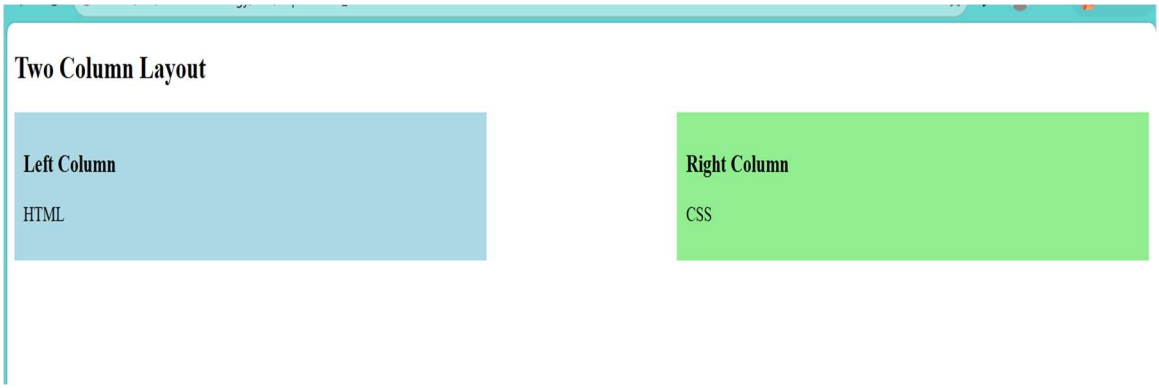
- Create a new HTML file.
- Type the XHTML program using the CSS float property.
- Save the file with the .html extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Two Column Layout</title>
  <style>
    .left{
      float:left;
      width:40%;
      background-color:lightblue;
      padding:10px;
    }
    .right{
      float:right;
      width:40%;
      background-color:lightgreen;
      padding:10px;
    }
  </style>
</head>
<body>
<h2>Two Column Layout</h2>
<div class="left">
<h3>Left Column</h3>
```

```
<p>HTML</p>
</div>
<div class="right">
<h3>Right Column</h3>
<p>CSS</p>
</div>
</body>
</html>
```

Expected Output



Week 7
Responsive Web Design

Experiment 25

Aim

To apply CSS media queries for responsive web design.

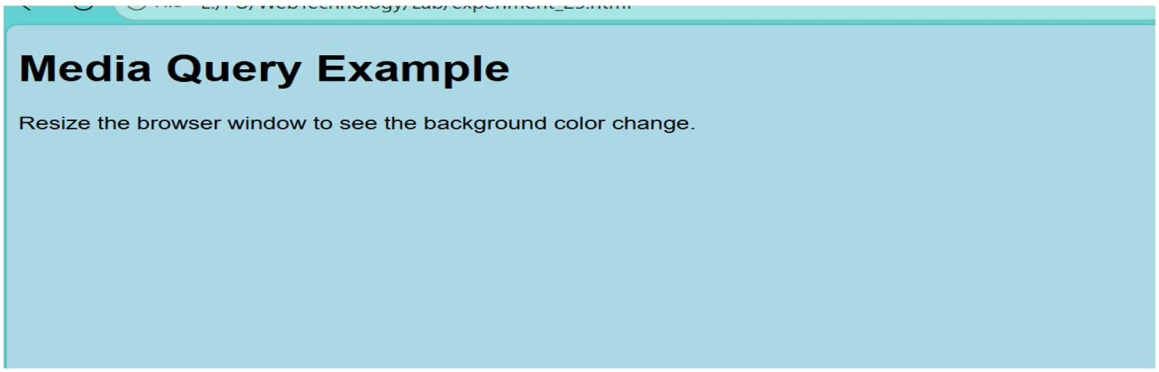
Procedure

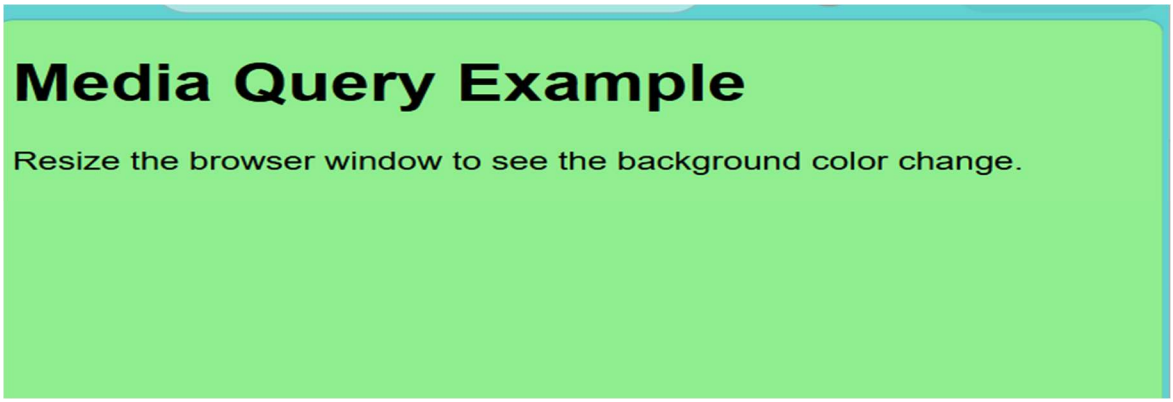
- Create a new HTML file.
- Type the XHTML program using CSS media queries.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Resize the browser window and verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Media Query Example</title>
  <style>
    body{
      background-color: lightblue;
      font-family: Arial;
    }
    @media screen and (max-width:600px){
      body{
        background-color: lightgreen;
      }
    }
  </style>
</head>
<body>
<h1>Media Query Example</h1>
<p>Resize the browser window to see the background color change.</p>
</body>
</html>
```

Expected Output





Experiment 26

Aim

To design a simple responsive webpage using CSS media queries.

Procedure

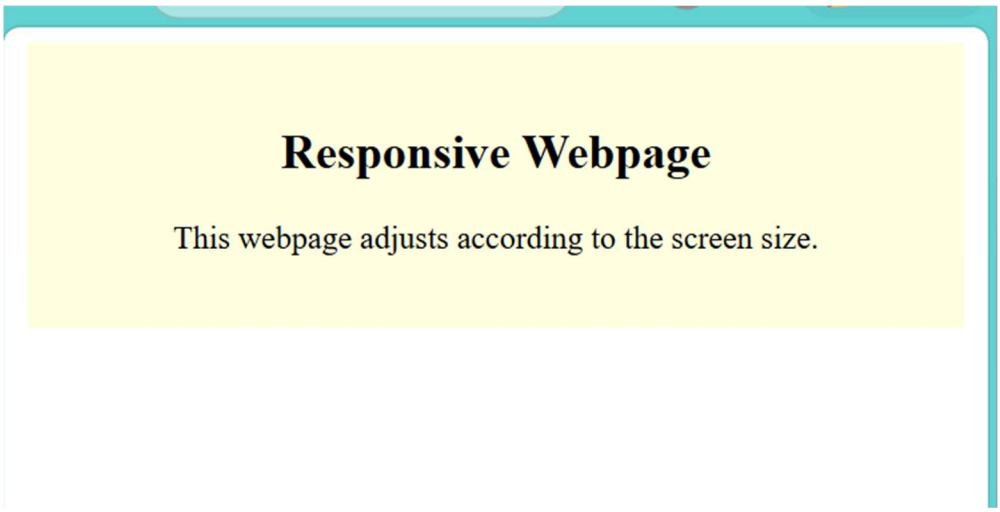
- Create a new HTML file.
- Type the XHTML program using CSS media queries.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Resize the browser window and verify the responsive layout.

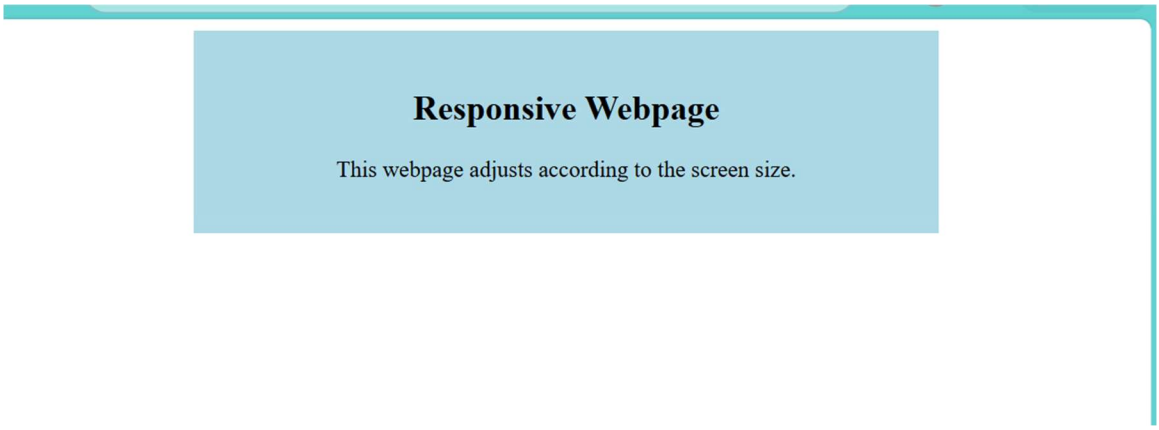
Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Responsive Webpage</title>
  <style>
    .box{
      width:60%;
      margin:auto;
      padding:20px;
      background-color:lightblue;
      text-align:center;
    }
    @media screen and (max-width:600px){
      .box{
        width:90%;
        background-color:lightyellow;
      }
    }
  </style>
</head>
<body>
<div class="box">
<h2>Responsive Webpage</h2>
<p>This webpage adjusts according to the screen size.</p>
</div>
```

```
</body>
</html>
```

Expected Output





CSS Frameworks

Experiment 27

Aim

To use Bootstrap framework components in a webpage.

Procedure

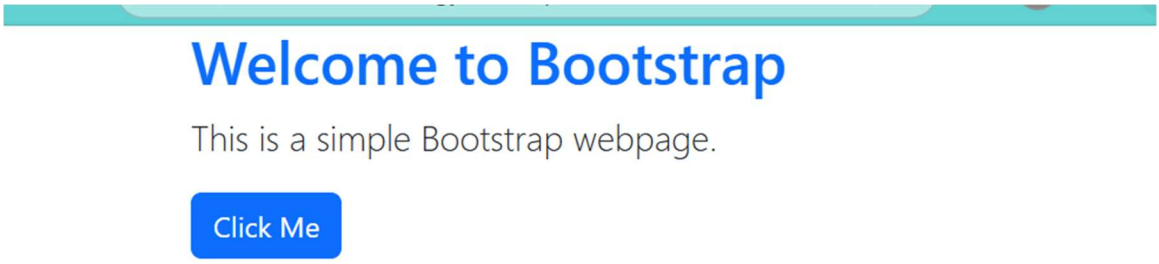
- Create a new HTML file.
- Add the Bootstrap CDN link.
- Use Bootstrap classes to style the webpage.
- Save the file with the **.html** extension.
- Open the file in a web browser and verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
<title>Bootstrap Basics</title>
```

```
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
</head>
<body>
<div class="container">
<h1 class="text-primary">Welcome to Bootstrap</h1>
<p class="lead">
This is a simple Bootstrap webpage.
</p>
<button class="btn btn-primary">
Click Me
</button>
</div>
</body>
</html>
```

Expected Output



Experiment 28

Aim

To design a responsive webpage using Bootstrap.

Procedure

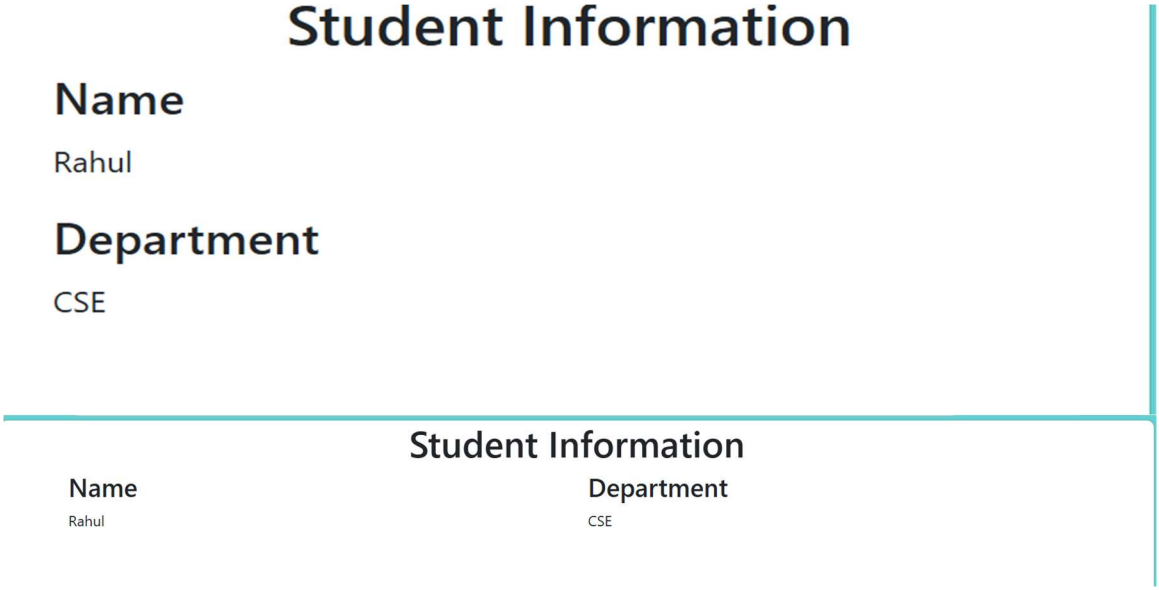
- Create a new HTML file.
- Add the Bootstrap CDN link.
- Use Bootstrap grid classes to create a responsive layout.
- Save the file with the **.html** extension.
- Open the file in a web browser and verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
<title>Responsive Bootstrap Page</title>
<link rel="stylesheet"
```

```
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
</head>
<body>
<div class="container">
<h1 class="text-center">Student Information</h1>
<div class="row">
<div class="col-md-6">
<h3>Name</h3>
<p>Rahul</p>
</div>
<div class="col-md-6">
<h3>Department</h3>
<p>CSE</p>
</div>
</div>
</div>
</body>
</html>
```

Expected Output



Week 8
XML Structure

Experiment 29

Aim

To create a simple XML document.

Procedure

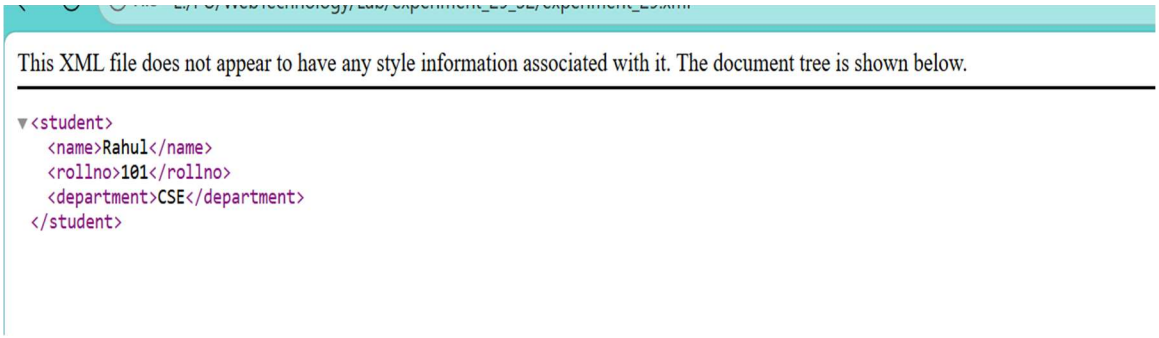
- Create a new file with the .xml extension.
- Type the XML program.
- Save the file.
- Open the XML file in a web browser.
- Verify the output.

Program (Source Code)

File: student.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<student>
  <name>Rahul</name>
  <rollno>101</rollno>
  <department>CSE</department>
</student>
```

Expected Output



Experiment 30

Aim

To display XML data in a webpage.

Procedure

- Create an XML file named student.xml.
- Create an HTML file named index.html.
- Write the XML and HTML code.
- Save both files in the same folder.
- Open the HTML file in a web browser and verify the output.

Program (Source Code)

File 1: student.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/css" href="style.css"?>

<students>

  <student>
    <name>Rahul</name>
    <rollno>101</rollno>
    <department>CSE</department>
  </student>

  <student>
    <name>Anitha</name>
    <rollno>102</rollno>
    <department>ECE</department>
  </student>

  <student>
    <name>Rohan</name>
    <rollno>103</rollno>
    <department>EEE</department>
  </student>

  <student>
    <name>Arjun</name>
    <rollno>105</rollno>
    <department>AIML</department>
  </student>

</students>
```

File 2: style.css

```
students{
  display:block;
  font-family:Arial;
  background-color:#f4f4f4;
  padding:15px;
}

student{
  display:block;
  border:2px solid blue;
  margin:10px;
  padding:10px;
  background-color:white;
}

name, rollno, department{
  display:block;
  margin:5px;
```

```
  font-size:18px;
}
```

Expected Output



XML Applications

Experiment 31

Aim

To validate the structure of an XML document.

Procedure

- Create a new file with the .xml extension.
- Type the XML code with proper opening and closing tags.
- Save the file.
- Open the XML file in a web browser.
- Verify that the XML structure is displayed without errors.

Program (Source Code)

File: student.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<employees>

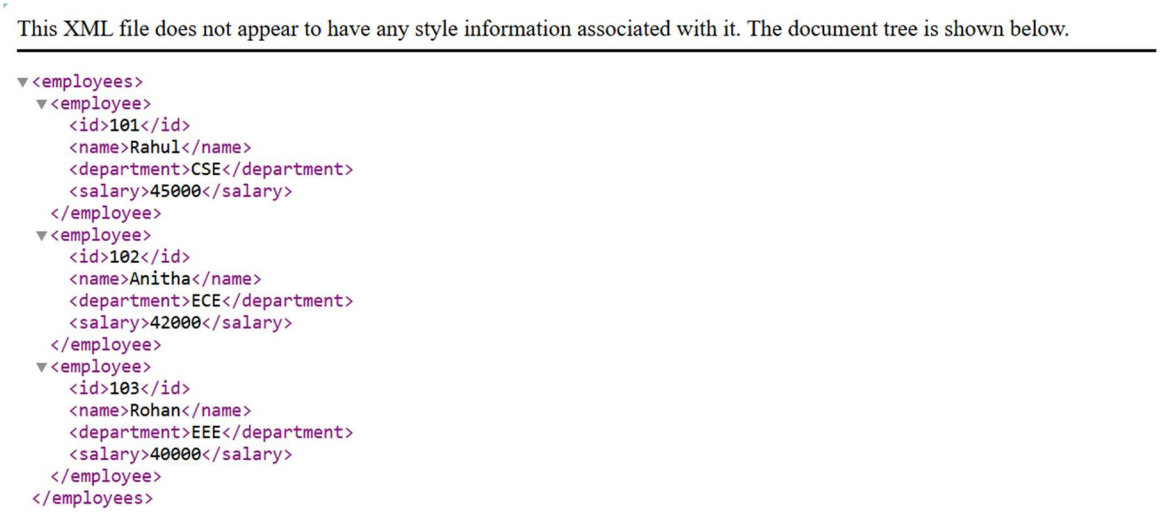
  <employee>
    <id>101</id>
    <name>Rahul</name>
    <department>CSE</department>
    <salary>45000</salary>
  </employee>
```

```
  <employee>
    <id>102</id>
    <name>Anitha</name>
    <department>ECE</department>
    <salary>42000</salary>
  </employee>

  <employee>
    <id>103</id>
    <name>Rohan</name>
    <department>EEE</department>
    <salary>40000</salary>
  </employee>

</employees>
```

Expected Output



(The browser displays the XML document in a tree structure without any errors, indicating that the XML is well-formed.)

Experiment 32

Aim

To use XML for data representation in web pages.

Procedure

- Create an XML file named **student.xml.**
- Create an HTML file named **index.html.**
- Save both files in the same folder.
- Open the HTML file in a web browser.
- Verify the displayed data.

Program (Source Code)

File 1: student.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/css" href="style1.css"?>

<employees>

  <employee>
    <id>101</id>
    <name>Rahul</name>
    <department>CSE</department>
    <salary>45000</salary>
  </employee>

  <employee>
    <id>102</id>
    <name>Anitha</name>
    <department>ECE</department>
    <salary>42000</salary>
  </employee>

  <employee>
    <id>103</id>
    <name>Rohan</name>
    <department>EEE</department>
    <salary>40000</salary>
  </employee>

</employees>
```

For the same above experiment

File 2: style1.css

```
employees{
  display:block;
  font-family:Arial;
  background-color:lightyellow;
  padding:20px;
}

employee{
  display:block;
  border:2px solid blue;
  background-color:white;
  padding:10px;
  margin:10px;
}

id,name,department,salary{
  display:block;
  padding:5px;
}
```

Expected Output

```
101
Rahul
CSE
45000
```

```
102
Anitha
ECE
42000
```

```
103
Rohan
EEE
40000
```

Week 9
JavaScript Introduction

Experiment 33

Aim

To write a basic JavaScript program for displaying output on a webpage.

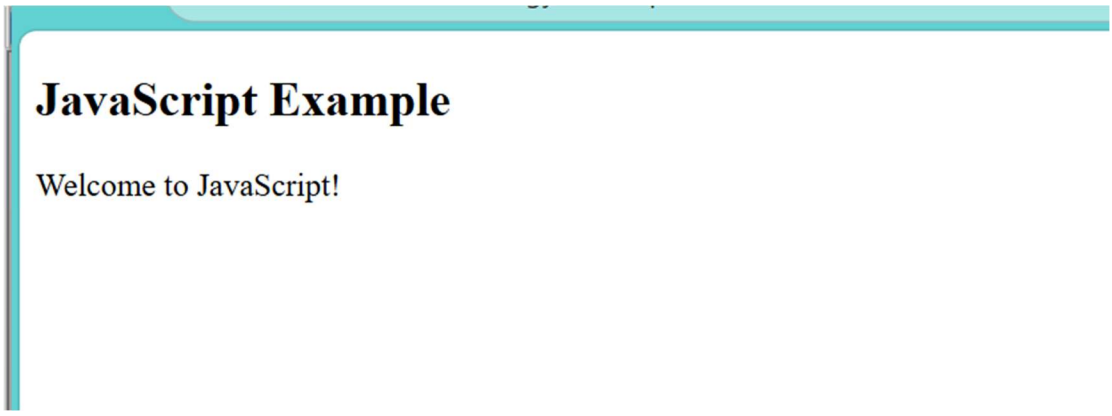
Procedure

- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
   <title>JavaScript Output</title>
</head>
<body>
<h2>JavaScript Example</h2>
<script>
document.write("Welcome to JavaScript!");
</script>
</body>
</html>
```

Expected Output



Experiment 34

Aim

To perform arithmetic operations using JavaScript.

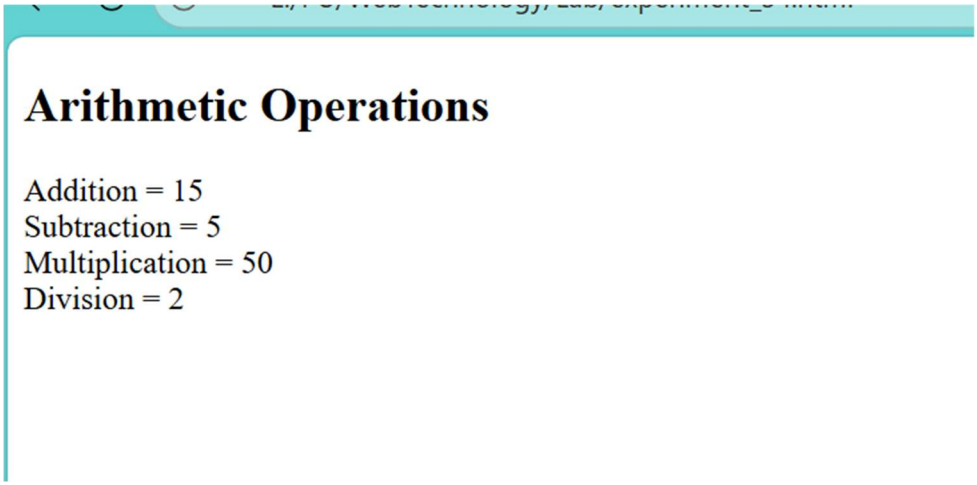
Procedure

- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
   <title>Arithmetic Operations</title>
</head>
<body>
<h2>Arithmetic Operations</h2>
<script>
let a = 10;
let b = 5;
document.write("Addition = " + (a + b) + "<br>");
document.write("Subtraction = " + (a - b) + "<br>");
document.write("Multiplication = " + (a * b) + "<br>");
document.write("Division = " + (a / b));
</script>
</body>
</html>
```

Expected Output



Control Statements

Experiment 35

Aim

To use **if-else** statements in JavaScript.

Procedure

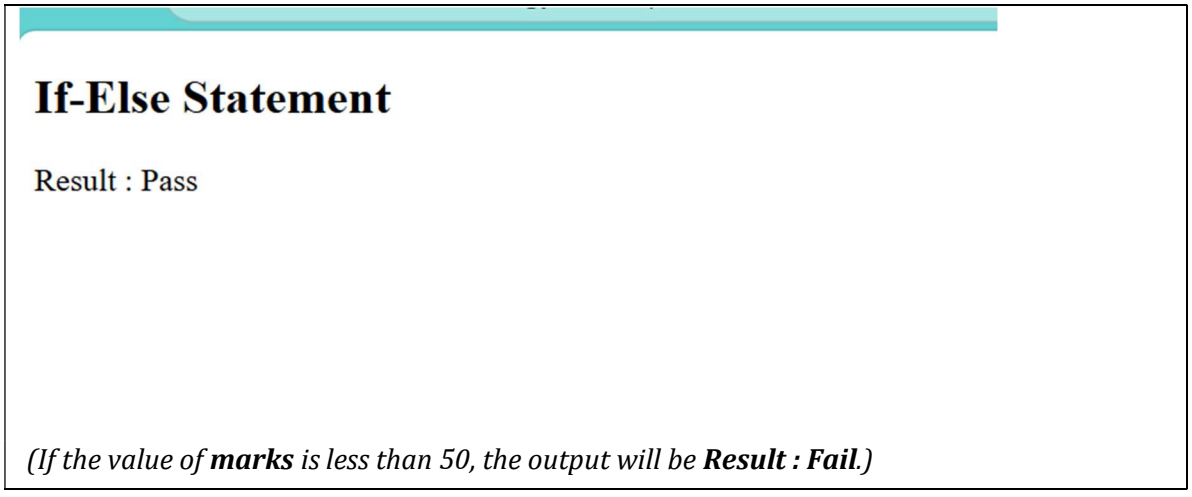
- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
   <title>If-Else Example</title>
</head>
<body>
<h2>If-Else Statement</h2>

<script>
let marks = 75;
if(marks >= 50)
{
   document.write("Result : Pass");
}
else
{
   document.write("Result : Fail");
}
</script>
</body></html>
```

Expected Output



*(If the value of **marks** is less than 50, the output will be **Result : Fail**.)*

Experiment 36

Aim

To implement loops in JavaScript.

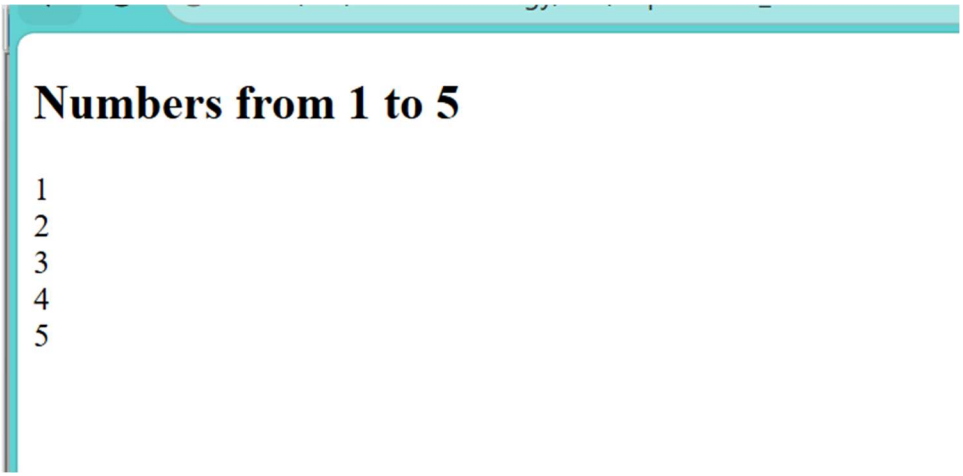
Procedure

- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
   <title>For Loop Example</title>
</head>
<body>
<h2>Numbers from 1 to 5</h2>
<script>
for(let i = 1; i <= 5; i++)
{
   document.write(i + "<br>");
}
</script>
</body></html>
```

Expected Output



Week 10
Functions

Experiment 37

Aim

To create and call functions in JavaScript.

Procedure

- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Function</title>
</head>
<body>
<h2>Function Example</h2>
<script>
function greet()
{
  document.write("Welcome to JavaScript!");
}
greet();
</script>
</body>
</html>
```

Expected Output



Experiment 38

Aim

To use JavaScript functions with parameters and return values.

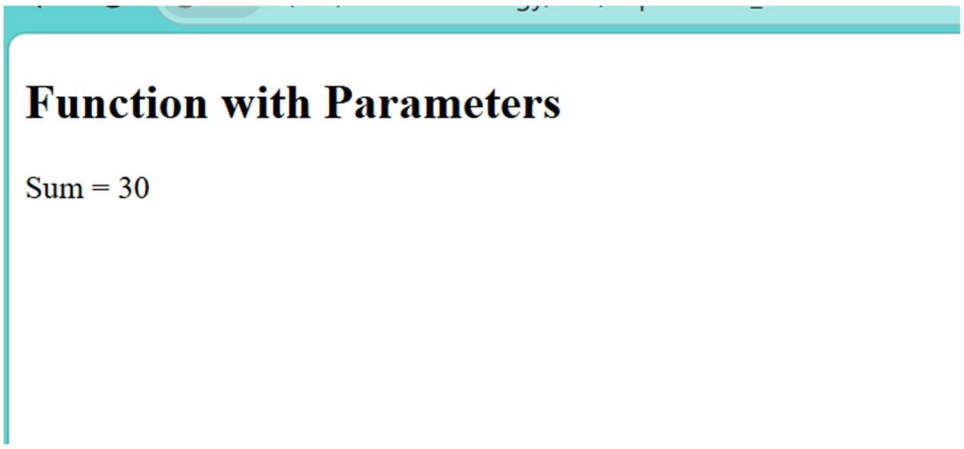
Procedure

- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Function with Parameters</title>
</head>
<body>
<h2>Function with Parameters</h2>
<script>
function add(a, b)
{
  return a + b;
}
let result = add(10, 20);
document.write("Sum = " + result);
</script>
</body>
</html>
```

Expected Output



DOM Manipulation

Experiment 39

Aim

To access HTML elements using the Document Object Model (DOM) in JavaScript.

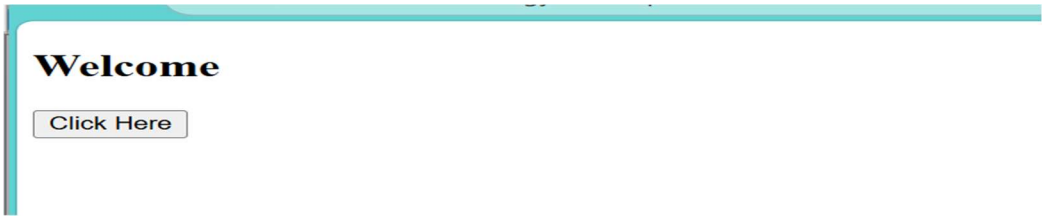
Procedure

- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Click the button and verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>DOM Example</title>
</head>
<body>
<h2 id="msg">Welcome</h2>
<button onclick="showMessage()">
Click Here
</button>
<script>
function showMessage()
{
  document.getElementById("msg").innerHTML = "Welcome to JavaScript DOM!";
}
</script>
</body>
</html>
```

Expected Output





Experiment 40

Aim

To modify webpage content dynamically using JavaScript.

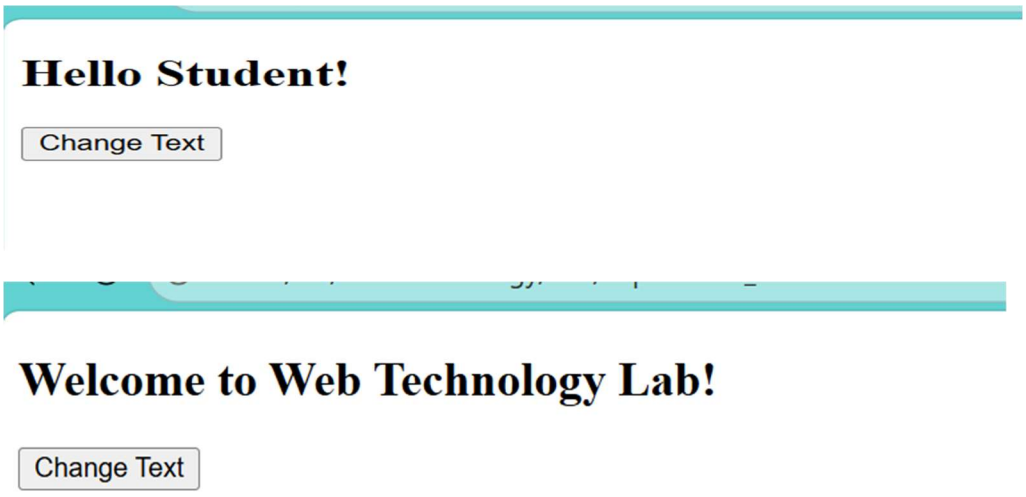
Procedure

- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Click the button and verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Dynamic Content</title>
</head>
<body>
<h2 id="text">Hello Student!</h2>
<button onclick="changeText()">
Change Text
</button>
<script>
function changeText()
{
  document.getElementById("text").innerHTML = "Welcome to Web Technology Lab!";
}
</script>
</body>
</html>
```

Expected Output



Week 11
Event Handling

Experiment 41

Aim

To handle click events using JavaScript.

Procedure

- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Click the button and verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Click Event</title>
</head>
<body>
<h2>Click Event Example</h2>
<button onclick="showMessage()">
Click Me
</button>
<script>
function showMessage()
{
  alert("Button Clicked!");
}
</script></body></html>
```

Expected Output



Experiment 42

Aim

To create an interactive webpage using JavaScript events.

Procedure

- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Perform the events and verify the output.

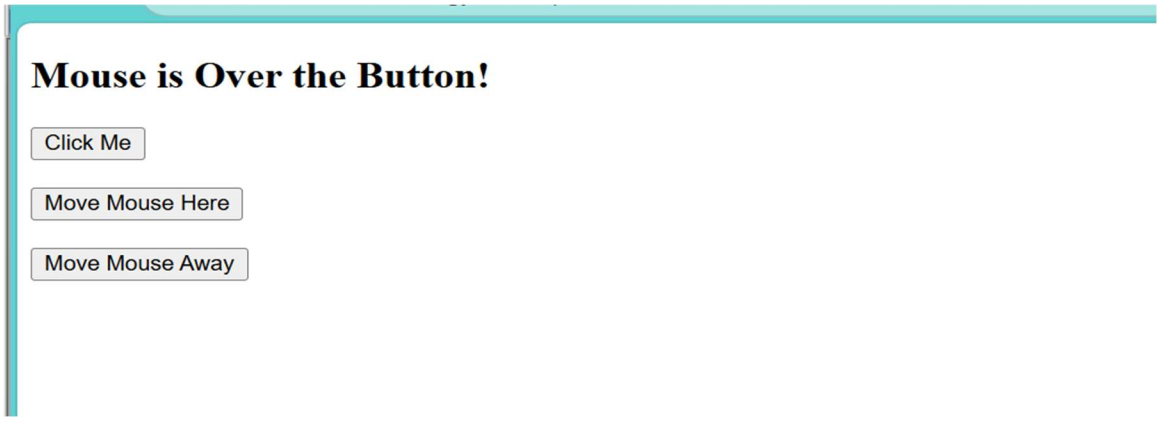
Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Events</title>
</head>
<body>
<h2 id="msg">Welcome to Web Technology Lab</h2>
<button onclick="clickEvent()">
Click Me
</button>
<br><br>
<button onmouseover="mouseOverEvent()">
Move Mouse Here
</button>
<br><br>
<button onmouseout="mouseOutEvent()">
Move Mouse Away
</button>
<script>
function clickEvent()
{
  document.getElementById("msg").innerHTML =
  "Button Clicked!";
}

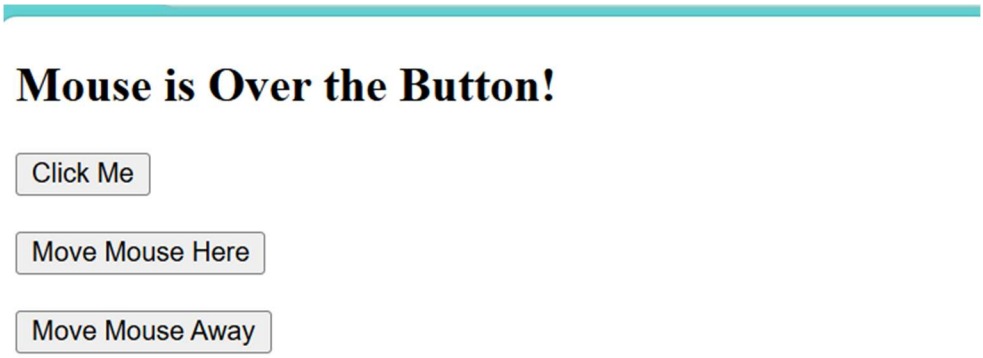
function mouseOverEvent()
{
  document.getElementById("msg").innerHTML =
  "Mouse is Over the Button!";
}

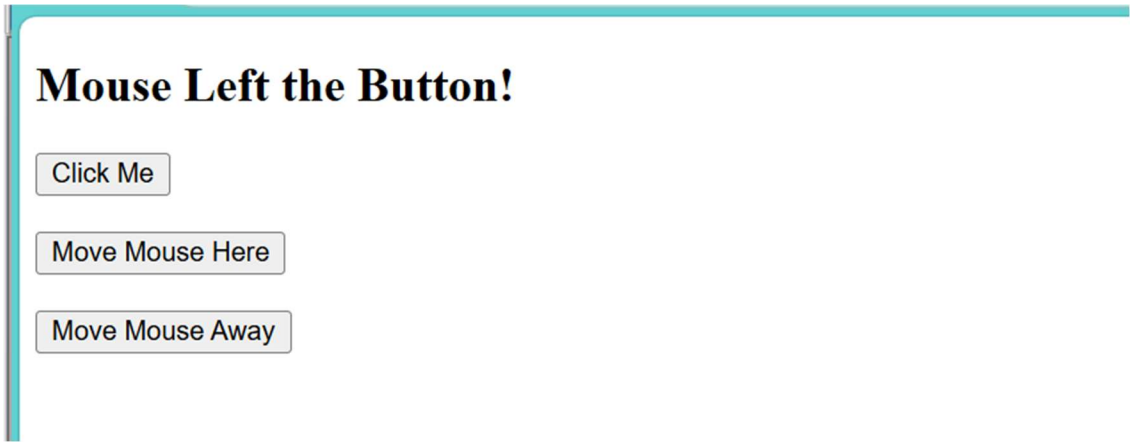
function mouseOutEvent()
{
  document.getElementById("msg").innerHTML =
  "Mouse Left the Button!";
}
</script>
</body>
</html>
```

Expected Output









Form Validation

Experiment 43

Aim

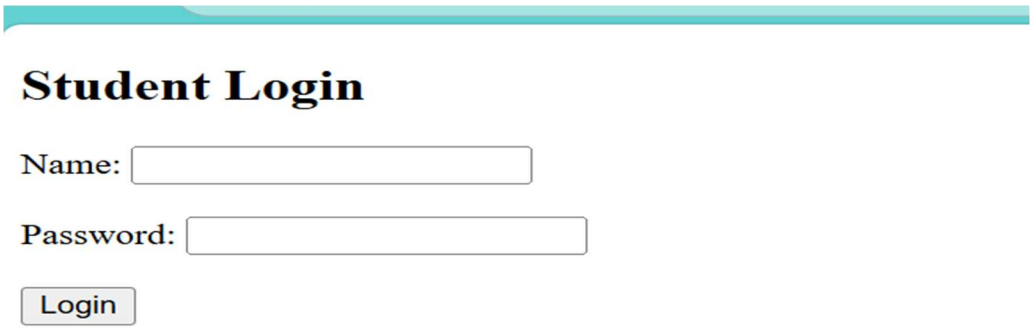
To validate input fields using JavaScript.

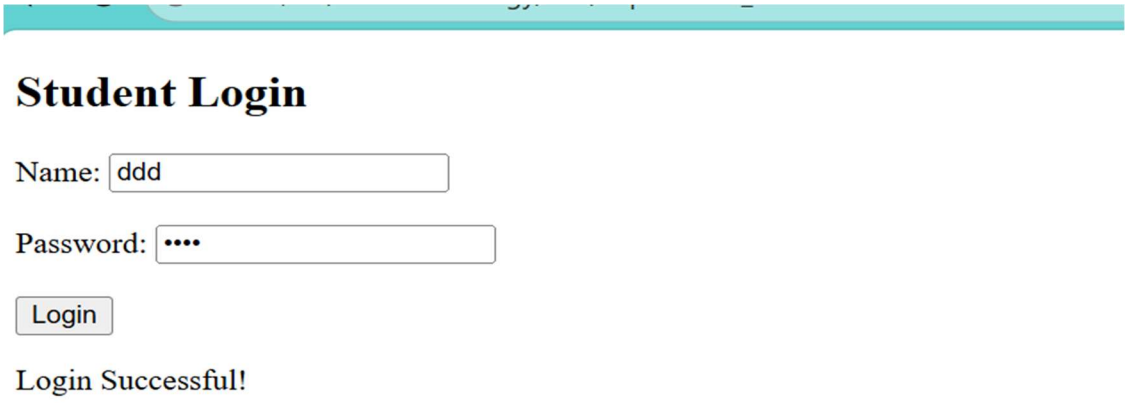
Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Input Validation</title>
</head>
<body>
<h2>Student Login</h2>
Name:
<input type="text" id="name"><br><br>
Password:
<input type="password" id="password"><br><br>
<button onclick="validate()">Login</button>
<p id="msg"></p>
<script>
function validate()
{
  let name=document.getElementById("name").value;
  let password=document.getElementById("password").value;

  if(name=="" || password=="")
    document.getElementById("msg").innerHTML="Please fill all fields.";
  else
    document.getElementById("msg").innerHTML="Login Successful!";
}
</script>
</body>
</html>
```

Expected Output







Experiment 44

Aim

To create a complete form validation system using JavaScript.

Program (Source Code)

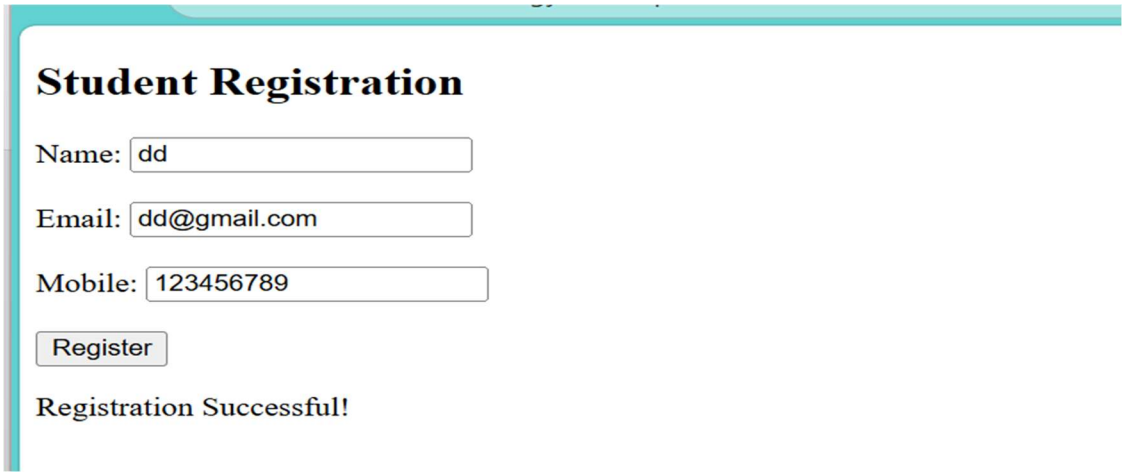
```
<!DOCTYPE html>
<html>
<head>
  <title>Registration Form</title>
</head>
<body>
<h2>Student Registration</h2>
Name:
<input type="text" id="name"><br><br>
Email:
<input type="text" id="email"><br><br>
Mobile:
<input type="text" id="mobile"><br><br>
<button onclick="validateForm()">Register</button>
<p id="msg"></p>
<script>
function validateForm()
{
```

```
  let name=document.getElementById("name").value;
  let email=document.getElementById("email").value;
  let mobile=document.getElementById("mobile").value;

  if(name=="" || email=="" || mobile=="")
    document.getElementById("msg").innerHTML="Please fill all the fields.";
  else
    document.getElementById("msg").innerHTML="Registration Successful!";
}
</script>
</body>
</html>
```

Expected Output





Week 12
Objects & Methods

Experiment 45

Aim

To create JavaScript objects.

Procedure

- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

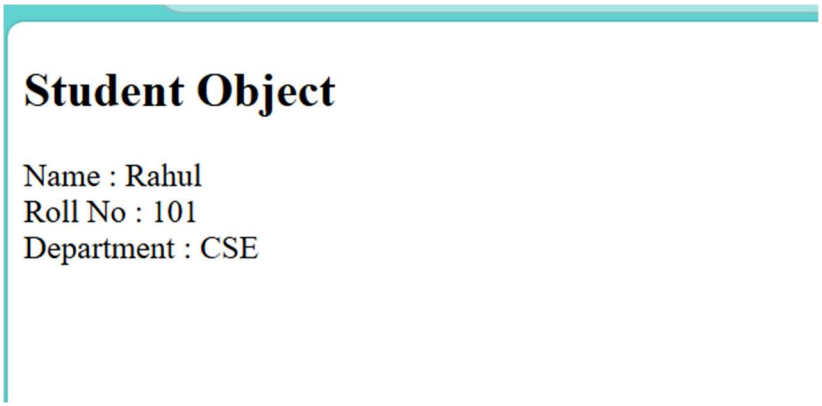
Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Object</title>
</head>
<body>
<h2>Student Object</h2>
<script>
let student = {
  name: "Rahul",
  rollNo: 101,
  department: "CSE"
};

document.write("Name : " + student.name + "<br>");
document.write("Roll No : " + student.rollNo + "<br>");
document.write("Department : " + student.department);

</script>
</body>
</html>
```

Expected Output



Experiment 46

Aim

To use JavaScript object methods and properties.

Procedure

- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Verify the output.

Program (Source Code)

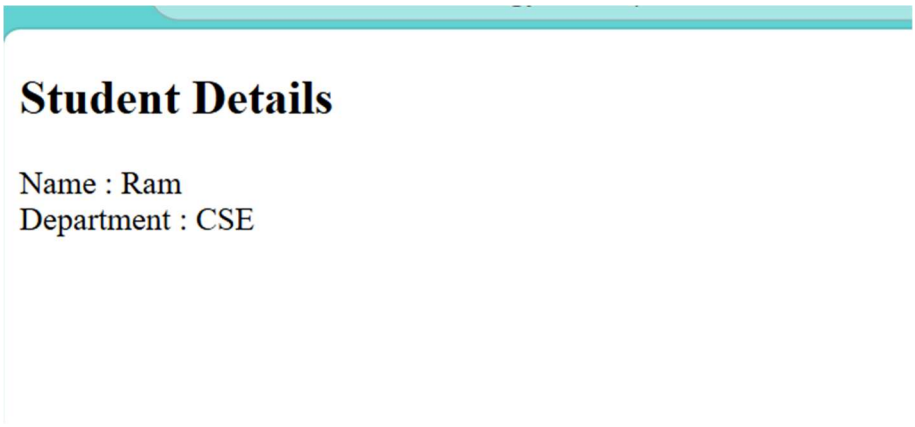
```
<!DOCTYPE html>
<html>
<head>
  <title>Object Methods</title>
</head>
<body>
<h2>Student Details</h2>
<script>
let student = {

  name: "Rahul",
  department: "CSE",

  display: function()
  {
    return "Name : " + this.name +
        "<br>Department : " + this.department;
  }

};
document.write(student.display());
</script>
</body>
</html>
```

Expected Output



Pop-ups

Experiment 47

Aim

To implement **alert()** and **prompt()** methods using JavaScript.

Procedure

- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Click the button and verify the output.

Program (Source Code)

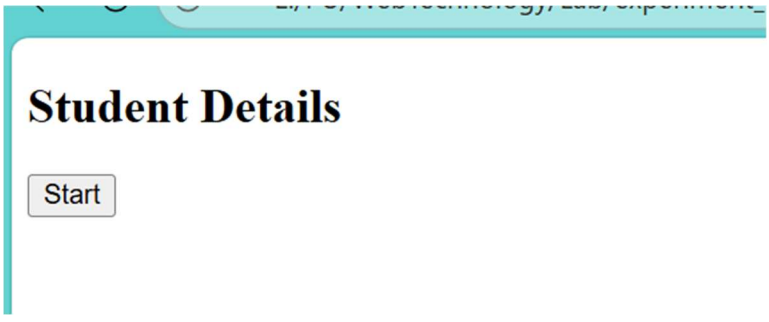
```
<!DOCTYPE html>
<html>
<head>
  <title>Alert and Prompt</title>
</head>
<body>
<h2>Student Details</h2>
<button onclick="showMessage()">
Start
</button>
<script>
function showMessage()
{
  alert("Welcome to Web Technology Lab!");

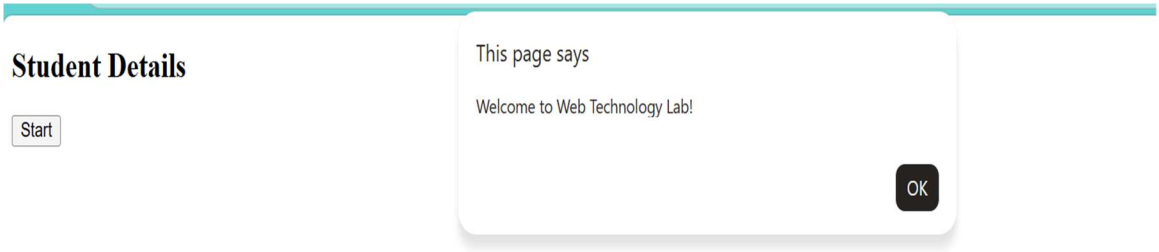
  let name = prompt("Enter your Name:");

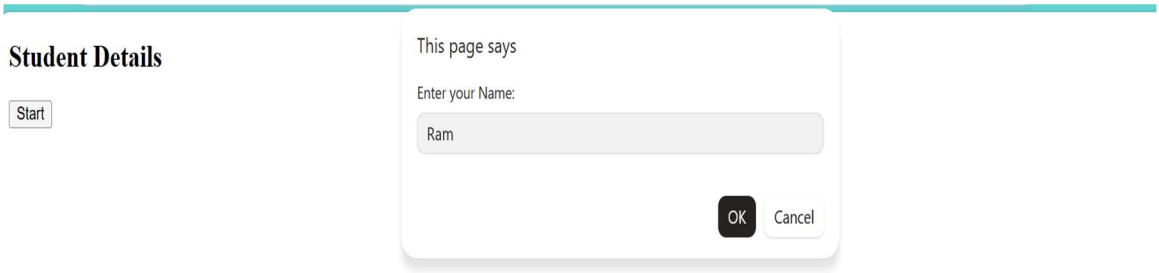
  alert("Hello " + name);

  alert("Thank You!");
}
</script>
</body>
</html>
```

Expected Output









Experiment 48

Aim

To create a dynamic pop-up-based interaction using JavaScript.

Procedure

- Create a new HTML file.
- Type the HTML and JavaScript code.
- Save the file with the **.html** extension.
- Open the file in a web browser.
- Click the button and verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Dynamic Popup</title>
</head>
<body>
<h2>Student Registration</h2>
<button onclick="register()">
Register
</button>
<p id="msg"></p>
<script>
```

```
function register()
{
  let name = prompt("Enter your Name:");
  let age = prompt("Enter your Age:");

  let choice = confirm("Do you want to submit the details?");

  if(choice)
  {
    alert("Registration Successful!");

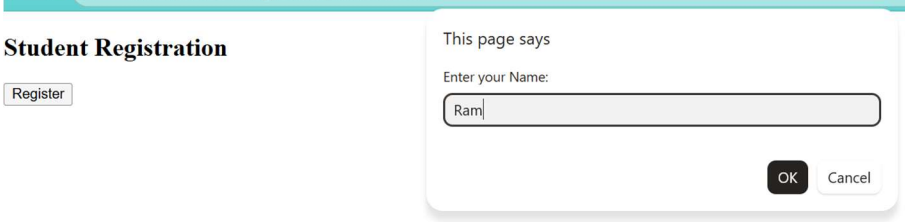
    document.getElementById("msg").innerHTML =
    "Name : " + name +
    "<br>Age : " + age;
  }
  else
  {
    alert("Registration Cancelled!");

    document.getElementById("msg").innerHTML =
    "No details submitted.";
  }
}

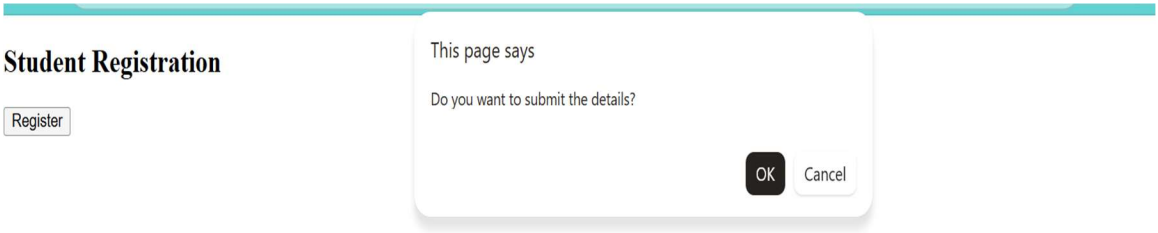
</script>
</body>
</html>
```

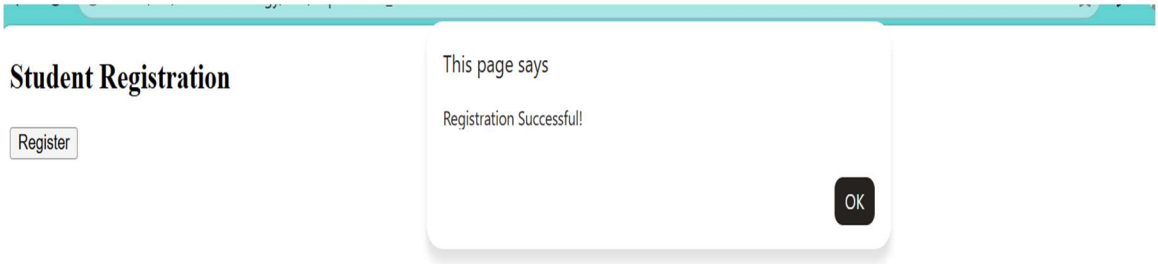
Expected Output











Student Registration

Register

Name : Ram
Age : 20

Week 13
PHP Introduction

Experiment 49

Aim

To write a basic PHP script.

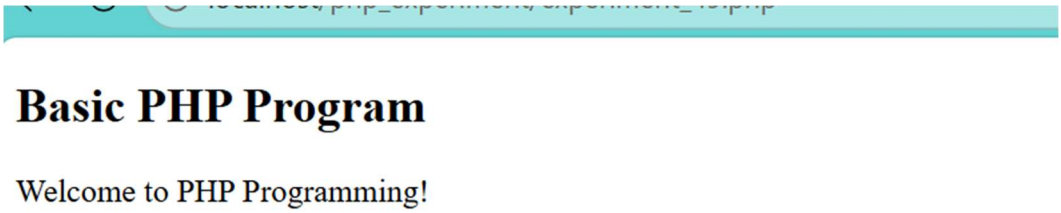
Procedure

- Open VS Code or any PHP editor.
- Create a new file with the **.php** extension.
- Type the PHP code.
- Save the file.
- Run the program using a local server (XAMPP/WAMP) and verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Basic PHP</title>
</head>
<body>
<h2>Basic PHP Program</h2>
<?php
echo "Welcome to PHP Programming!";
?>
</body>
</html>
```

Expected Output



Experiment 50

Aim

To handle form input using PHP.

Procedure

- Create a PHP file.
- Type the HTML form and PHP code.
- Save the file with the **.php** extension.
- Run the program using XAMPP/WAMP.
- Enter the details and verify the output.

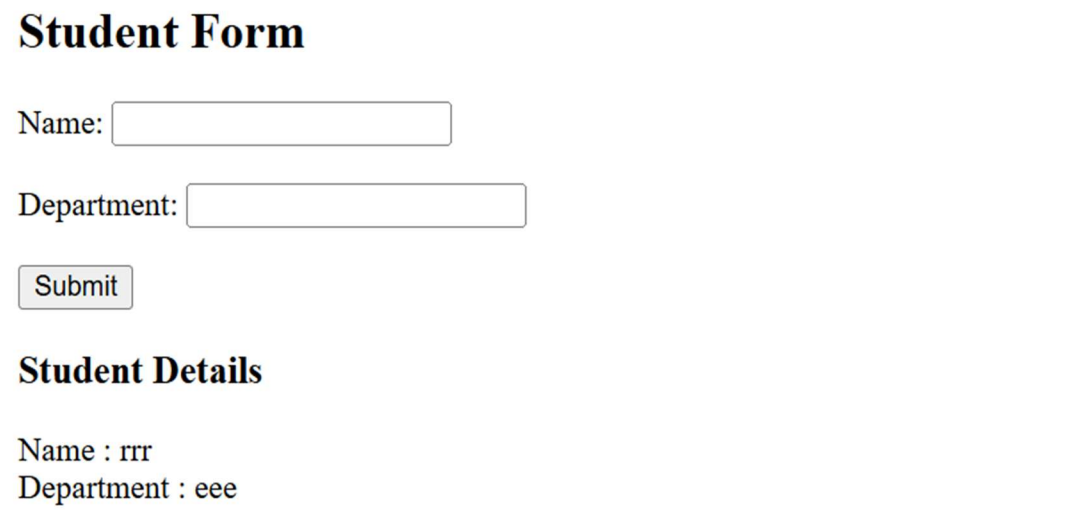
Program (Source Code)

```
<!DOCTYPE html>
<html>

<head>
  <title>PHP Form</title>
</head>
<body>
<h2>Student Form</h2>
<form method="post">
Name:
<input type="text" name="name"><br><br>
Department:
<input type="text" name="dept"><br><br>
<input type="submit" value="Submit">
</form>
<?php
if($_SERVER["REQUEST_METHOD"]=="POST")
{
  $name=$_POST["name"];
  $dept=$_POST["dept"];

  echo "<h3>Student Details</h3>";
  echo "Name : ".$name."<br>";
  echo "Department : ".$dept;
}
?></body></html>
```

Expected Output



PHP Control Structures

Experiment 51

Aim

To use **if-else** statements in PHP.

Procedure

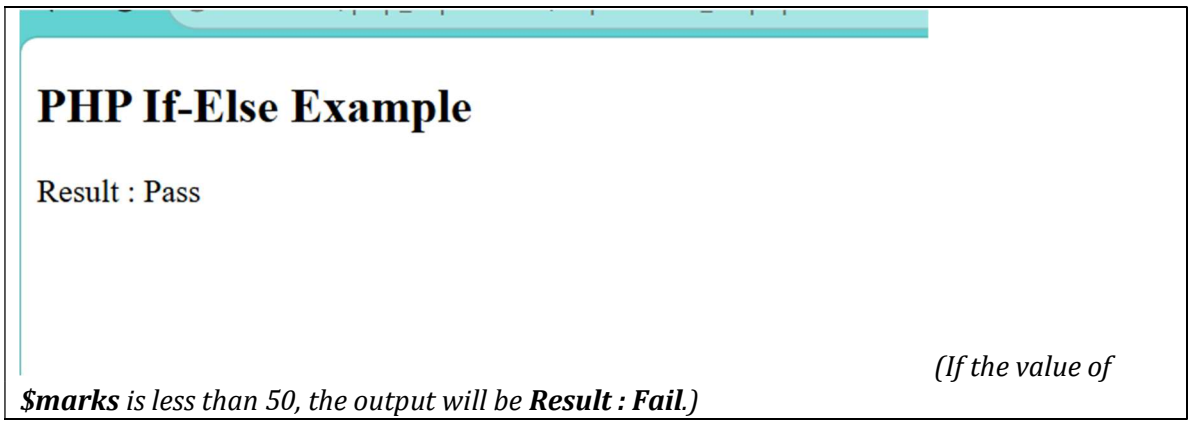
- Create a new file with the **.php** extension.
- Type the PHP code.
- Save the file.
- Run the program using XAMPP/WAMP.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>PHP If-Else</title>
</head>
<body>
<h2>PHP If-Else Example</h2>
<?ph
$marks = 75;
if($marks >= 50)
{
  echo "Result : Pass";
}
else
{
  echo "Result : Fail";
}

?>
</body>
</html>
```

Expected Output



*(If the value of **$marks** is less than 50, the output will be **Result : Fail**.)*

Experiment 52

Aim

To implement loops in PHP.

Procedure

- Create a new file with the **.php** extension.
- Type the PHP code.
- Save the file.
- Run the program using XAMPP/WAMP.
- Verify the output.

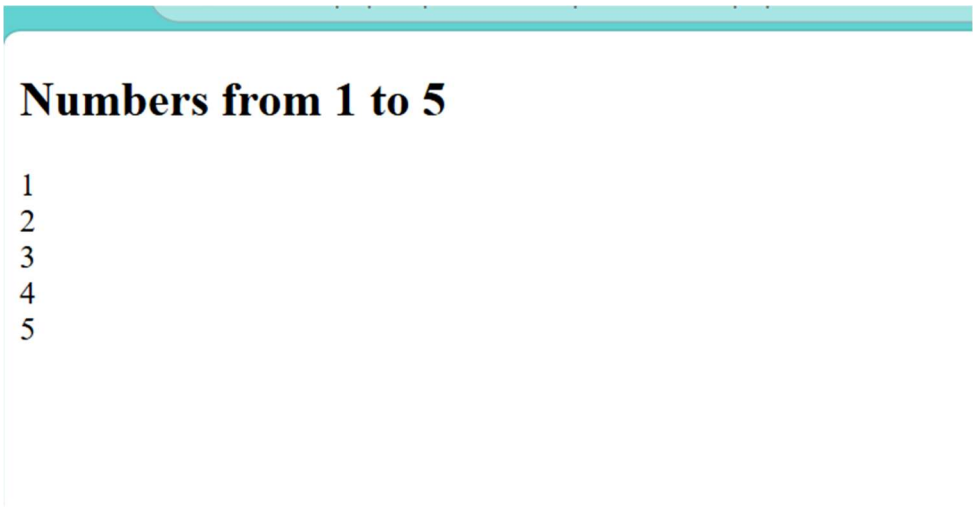
Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>PHP For Loop</title>
</head>

<body>
<h2>Numbers from 1 to 5</h2>
<?php
for($i = 1; $i <= 5; $i++)
{
  echo $i . "<br>";
}

?>
</body>
</html>
```

Expected Output



Week 14
PHP Forms

Experiment 53

Aim

To process form data using PHP.

Procedure

- Create a new file with the **.php** extension.
- Type the HTML form and PHP code.
- Save the file.
- Run the program using XAMPP/WAMP.
- Enter the details and verify the output.

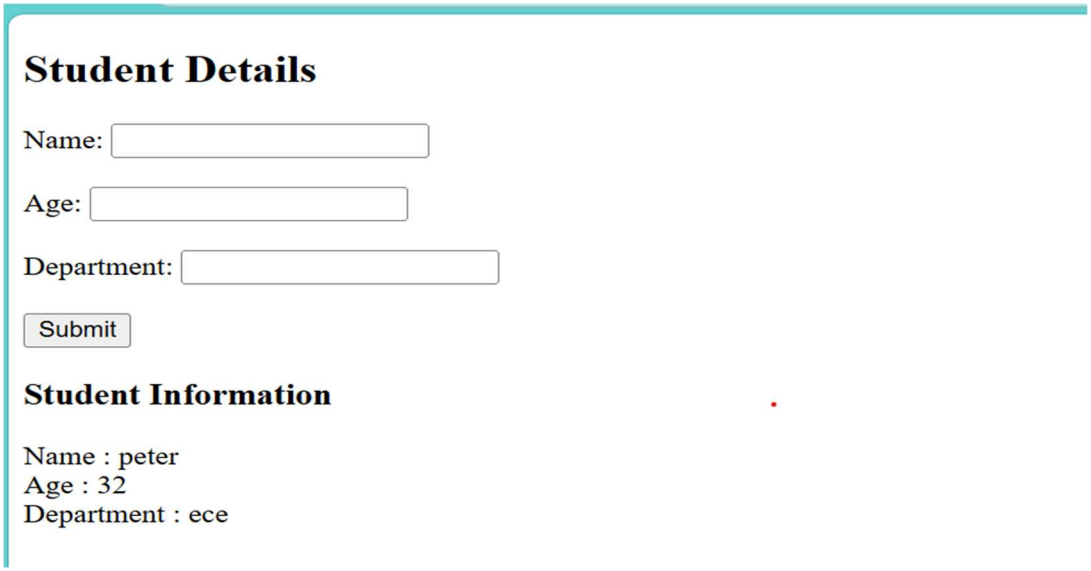
Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Process Form Data</title>
</head>
<body>
<h2>Student Details</h2>
<form method="post">
Name:
<input type="text" name="name"><br><br>
Age:
<input type="number" name="age"><br><br>
Department:
<input type="text" name="dept"><br><br>
<input type="submit" value="Submit">
</form>
<?php
if($_SERVER["REQUEST_METHOD"]=="POST")
{
  $name = $_POST["name"];
  $age = $_POST["age"];
  $dept = $_POST["dept"];

  echo "<h3>Student Information</h3>";
  echo "Name : ".$name."<br>";
  echo "Age : ".$age."<br>";
  echo "Department : ".$dept;
}

?>
</body>
</html>
```

Expected Output



Experiment 54

Aim

To create a simple login system using PHP.

Procedure

- Create a new file with the **.php** extension.
- Type the HTML form and PHP code.
- Save the file.
- Run the program using XAMPP/WAMP.
- Enter the login credentials and verify the output.

Program (Source Code)

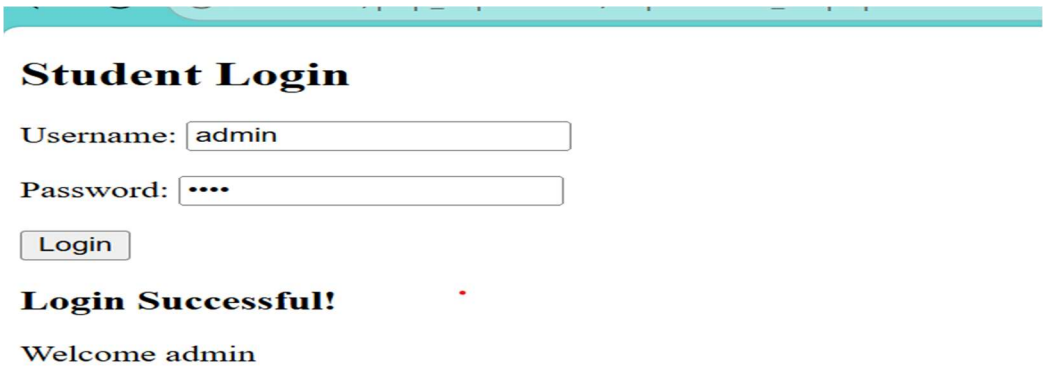
```
<!DOCTYPE html>
<html>
<head>
  <title>PHP Login</title>
</head>
<body>
<h2>Student Login</h2>
<form method="post">
Username:
<input type="text" name="username"><br><br>
Password:
<input type="password" name="password"><br><br>
<input type="submit" value="Login">
</form>
<?php
if($_SERVER["REQUEST_METHOD"]=="POST")
{
  $username = $_POST["username"];
  $password = $_POST["password"];
```

```
  if($username=="admin" && $password=="1234")
  {
    echo "<h3>Login Successful!</h3>";
    echo "Welcome ".$username;
  }
  else
  {
    echo "<h3>Invalid Username or Password!</h3>";
  }
}

?>
</body>
</html>
```

Expected Output





PHP File Handling

Experiment 55

Aim

To read and write data into files using PHP.

Procedure

- Create a new file with the .php extension.
- Write PHP code to create and open a file.
- Use file handling functions to write data into the file.
- Read the stored data from the file.
- Save the file in the XAMPP/WAMP server directory.
- Run the program in a web browser and verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
   <title>PHP File Handling</title>
</head>
<body>

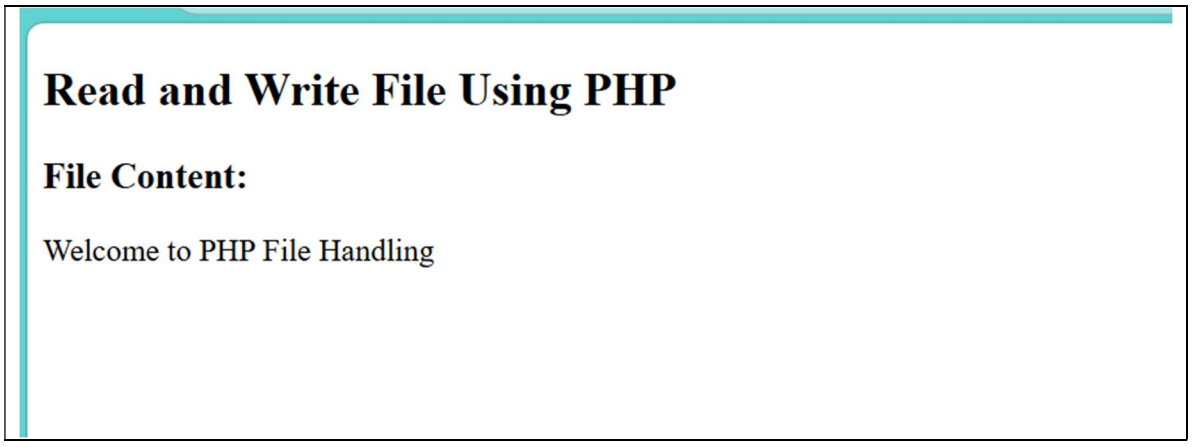
<h2>Read and Write File Using PHP</h2>

<?php
$file = fopen("data.txt","w");
fwrite($file,"Welcome to PHP File Handling");
fclose($file);
$file = fopen("data.txt","r");
$data = fread($file,filesize("data.txt"));
fclose($file);

echo "<h3>File Content:</h3>";
echo $data;
?>

</body>
</html>
```

Expected Output



Experiment 56

Aim

To store and retrieve data dynamically using PHP.

Procedure

- Create a new file with the .php extension.
- Create an HTML form to accept user details.
- Write PHP code to process the entered data.
- Store the data dynamically using PHP variables.
- Retrieve and display the stored data on the webpage.
- Save the file in the XAMPP/WAMP server directory.
- Run the program in a web browser and verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Dynamic Data Storage</title>
</head>
<body>
<h2>Student Details</h2>
<form method="post">
Name:
<input type="text" name="name"><br><br>
Department:
<input type="text" name="dept"><br><br>

<input type="submit" value="Submit">
</form>
<?php
if($_POST)
{
  $name = $_POST["name"];
  $dept = $_POST["dept"];

  echo "<h3>Student Details</h3>";
  echo "Name: ".$name."<br>";
  echo "Department: ".$dept;
}

?>
</body>
</html>
```

Expected Output

Initially, the webpage displays:
Student Details

Name: _____

Department: _____

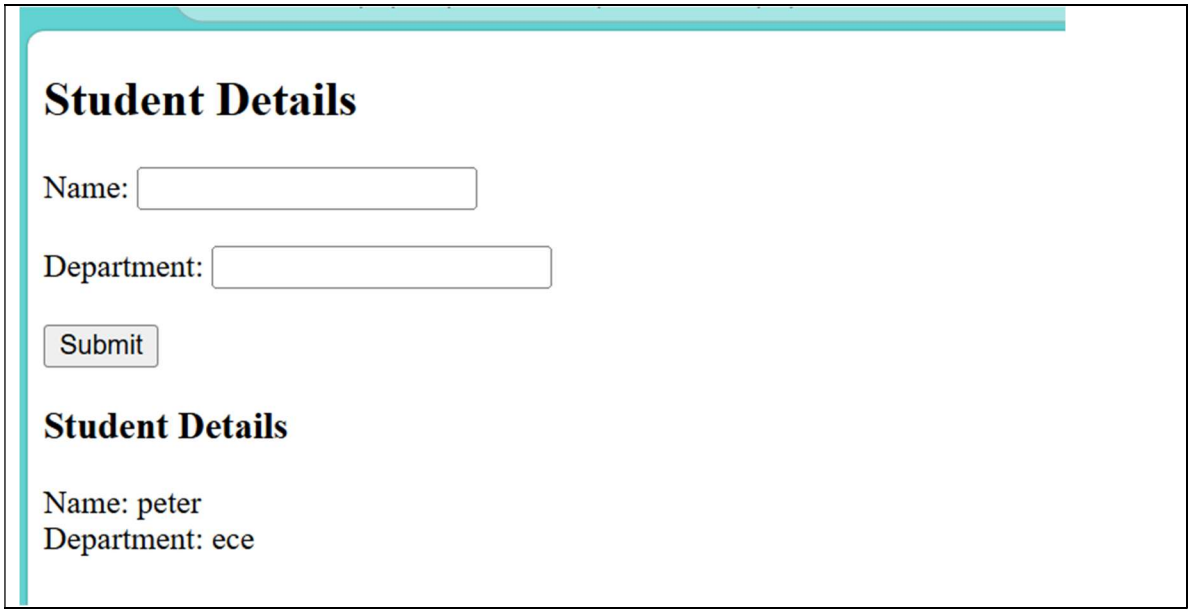
[Submit]

Input

Name: Rahul

Department: CSE

Output



Week 15

Web Integration

Experiment 57

Aim

To create a simple webpage by combining XHTML, CSS, and JavaScript.

Procedure

- Create a new file with the .html extension.
- Write the XHTML structure for creating the webpage.
- Add CSS styles to improve the appearance of the webpage.
- Add JavaScript code to perform dynamic operations.
- Save the file and open it in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>XHTML CSS JavaScript</title>
  <style>
    body {
      text-align: center;
      font-family: Arial;
    }
    h2 {
      color: blue;
    }
    button {
      padding: 10px;
    }
  </style>
</head>
<body>
<h2>Welcome to Web Technology</h2>
<p id="msg">Click the button to change the message</p>
<button onclick="changeMessage()">Click Here</button>
<script>
function changeMessage()
{
  document.getElementById("msg").innerHTML =
  "XHTML, CSS and JavaScript Combined Successfully!";
}
</script>
</body>
</html>
```

Expected Output





Experiment 58

Aim

To develop a dynamic student portfolio webpage by integrating XHTML, CSS, JavaScript, and PHP concepts learned throughout the Web Technology laboratory.

Procedure

- Create a new file with .php extension.
- Design the webpage structure using XHTML/HTML tags.
- Apply CSS styles to improve webpage appearance.
- Use JavaScript for dynamic content changes and form validation.
- Use PHP to process user input and display dynamic output.
- Save the file in the XAMPP/WAMP server directory.
- Run the program using a web browser and verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Student Portfolio</title>
<style>
body
{
   font-family: Arial;
   background-color: #f2f2f2;
   text-align: center;
}
.container
{
   width: 500px;
   margin: auto;
   background: white;
   padding: 20px;
   border: 1px solid black;
}
h1
{
```

```
   color: blue;
}
input, textarea
{
   width: 80%;
   padding: 8px;
   margin: 5px;
}
button
{
   padding: 10px;
}
#message
{
   color: green;
}
</style>
</head>
<body>
<div class="container">
<h1>Student Portfolio</h1>
<img src="student.jpg" width="120" height="120"
alt="Student Image"/>
<h2>About Me</h2>
<p>
I am a Computer Science student interested in Web Technologies.
</p>
<h2>Registration Form</h2>
<form method="post" onsubmit="return validate()">
Name:
<br>
<input type="text" name="name" id="name">
<br>
Email:
<br>
<input type="text" name="email" id="email">
<br>
Skills:
<br>
<textarea name="skill"></textarea>
<br>
<button type="submit">Submit</button>
</form>
<p id="message"></p>
<?php
if($_POST)
{
   $name = $_POST["name"];
   $email = $_POST["email"];
   $skill = $_POST["skill"];
   echo "<h2>Student Details</h2>";

   echo "Name : ".$name."<br>";
```

```
   echo "Email : ".$email."<br>";
   echo "Skills : ".$skill;
}
?>
</div>
<script>
function validate()
{
var name=document.getElementById("name").value;

var email=document.getElementById("email").value;

if(name=="" || email=="")
{
   document.getElementById("message").innerHTML =
   "Please fill all required fields";

   return false;
}

else
{
   alert("Form Submitted Successfully");

   return true;
}

}

</script>
</body>
</html>
```

Expected Output

Student Portfolio

[Student Image]

About Me

I am a Computer Science student interested in Web Technologies.

Registration Form

Name:
_____

Email:
_____

Skills:
_____

[Submit]

Case 1: Empty Fields

Input
Name: _____

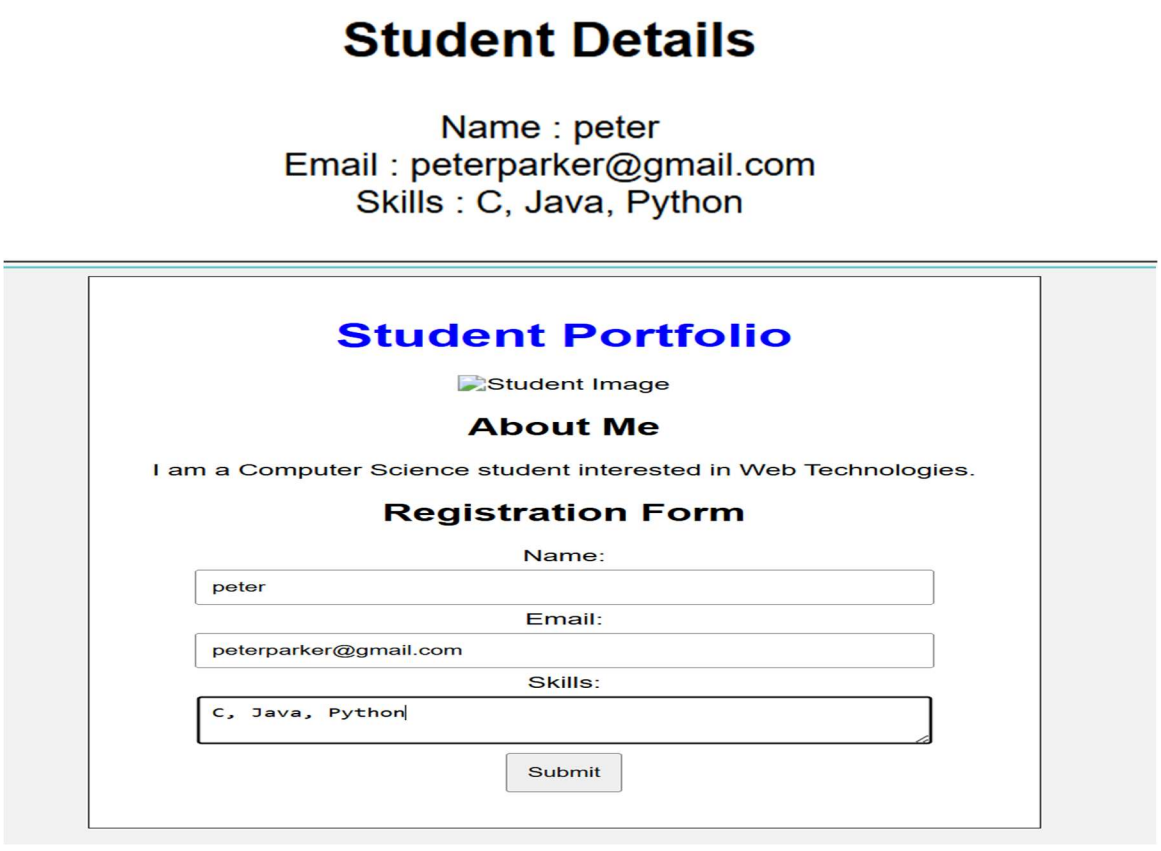
Email: _____
Output
Please fill all required fields

Case 2: Valid Input

Input
Name: Rahul

Email: rahul@gmail.com

Skills: HTML, CSS, JavaScript
Output
Form Submitted Successfully



Mini Project

Experiment 59

Aim

To design a simple and complete website using XHTML, CSS, and JavaScript.

Procedure

- Create a new file with the .html extension.
- Design the homepage using XHTML elements.
- Apply CSS to improve the appearance of the webpage.
- Add navigation links.
- Use JavaScript to display a welcome message.
- Save the file and open it in a web browser.
- Verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>My College Website</title>

<style>
body{
   font-family: Arial;
   background-color:#f2f2f2;
   margin:0;
}

header{
   background-color:steelblue;
   color:white;
   text-align:center;
   padding:15px;
}

nav{
   background-color:#333;
   text-align:center;
   padding:10px;
}

nav a{
   color:white;
   text-decoration:none;
   margin:15px;
}

section{
   padding:20px;
}
```

```
footer{
   background-color:steelblue;
   color:white;
   text-align:center;
   padding:10px;
}

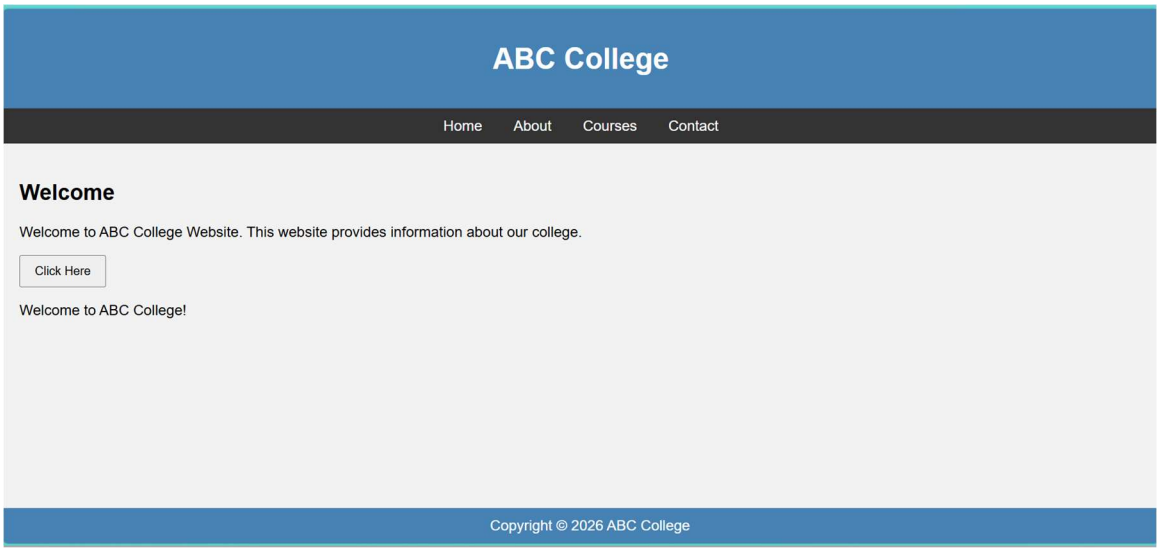
button{
   padding:8px 15px;
}

</style>
</head>
<body>
<header>
<h1>ABC College</h1>
</header>
<nav>
<a href="#">Home</a>
<a href="#">About</a>
<a href="#">Courses</a>
<a href="#">Contact</a>
</nav>
<section>

<h2>Welcome</h2>
<p>
Welcome to ABC College Website.
This website provides information about our college.
</p>
<button onclick="showMessage()">
Click Here
</button>
<p id="msg"></p>
</section>
<footer>
Copyright © 2026 ABC College
</footer>
<script>
function showMessage()
{
document.getElementById("msg").innerHTML=
"Welcome to ABC College!";
}

</script>
</body>
</html>
```

Expected Output



Experiment 60

Aim

To develop a simple web application using XHTML, CSS, JavaScript, and PHP.

Procedure

- Create a new file with the .php extension.
- Design the webpage using XHTML.
- Apply CSS for styling.
- Use JavaScript to validate the form.
- Use PHP to process and display the submitted data.
- Save the file in the XAMPP/WAMP server directory.
- Run the application in a web browser and verify the output.

Program (Source Code)

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Student Feedback</title>
<style>
body
{
   font-family: Arial;
   background-color: #f2f2f2;
}
.container
{
   width: 400px;
   margin: auto;
   background: white;
   padding: 20px;
   border: 1px solid black;
}
h2
```

```
{
   text-align: center;
}

input, textarea
{
   width: 100%;
   padding: 8px;
   margin-top: 5px;
   margin-bottom: 10px;
}

input[type=submit]
{
   width: auto;
   padding: 8px 15px;
}
</style>
<script>
function validate()
{
   var name = document.getElementById("name").value;
   var feedback = document.getElementById("feedback").value;

   if(name=="" || feedback=="")
   {
      alert("Please fill all fields");
      return false;
   }

   return true;
}
</script>
</head>
<body>
<div class="container">
<h2>Student Feedback Form</h2>
<form method="post" onsubmit="return validate();">
Name:
<input type="text" name="name" id="name">
Feedback:
<textarea name="feedback" id="feedback"></textarea>
<input type="submit" value="Submit">
</form>
<?php
if($_POST)
{
   $name = $_POST["name"];
   $feedback = $_POST["feedback"];

   echo "<h3>Feedback Submitted Successfully!</h3>";

   echo "Name : ".$name."<br><br>";
```

```
    echo "Feedback : ".$feedback;
}

?>
</div>
</body>
</html>
```

Expected Output

Initially, the webpage displays:
Student Feedback Form

Name:
_____

Feedback:
_____

[Submit]

Case 1: Empty Fields
Output
Please fill all fields

Case 2: Valid Input
Input
Name: Rahul

Feedback: Good Lab Session
Output
Feedback Submitted Successfully!

Name : Rahul

Feedback : Good Lab Session

